

UNIVERSIDAD NACIONAL DE ROSARIO
FACULTAD DE CIENCIAS EXACTAS, INGENIERÍA Y
AGRIMENSURA
Licenciatura en Ciencias de la Computación



UNR Universidad
Nacional de Rosario

Una infraestructura técnica y procedimental para el análisis de noticias de portales digitales a través de inteligencia artificial generativa

Tesina de grado

Trabajo presentado para optar al título de
Licenciado en Ciencias de la Computación

Autor: Lucio Bassani
Director: Alejandro Sartorio

Rosario, 2026

Índice general

1. Introducción	4
1.1. Motivación	4
1.2. Objetivos	6
2. Análisis bibliográfico?	8
2.1. Marco Teórico	8
2.2. Revisión Sistemática de la Literatura	11
2.2.1. Preguntas de Investigación	11
2.2.2. Metodología de Búsqueda y Selección	12
2.2.3. Estudios Incluidos	15
2.3. Estado del Arte	17
2.3.1. Limitaciones del Procesamiento de Lenguaje Natural (NLP) Tradicional	17
2.3.2. Aplicaciones Computacionales del Agenda-Setting de Segundo Nivel	17
2.3.3. Arquitecturas de Software con IAG para el Procesa- miento de Noticias	18
2.4. Síntesis Crítica y Brechas en la Investigación	19
2.5. Conclusión del Capítulo	20
3. Arquitectura	22
3.1. El estilo Tubos y Filtros	23
3.1.1. Componentes, conectores y patrón estructural	23
3.1.2. Modelo computacional e invariantes	24
3.1.3. Por qué Tubos y Filtros para el procesamiento gene- rativo de noticias	26
3.2. El framework como sistema de Tubos y Filtros	29
3.2.1. Vista de conjunto y patrón estructural	29

3.2.2.	Componentes: los filtros del framework	32
3.2.3.	Conectores: tubos tipados	39
3.2.4.	Especializaciones aplicadas: pipelines y tubos tipados .	41
3.2.5.	Deformación aplicada: repositorio común sobre Post- greSQL	42
3.2.6.	Sintonizadores	43
3.2.7.	Verificación de los invariantes esenciales	44
3.2.8.	Ventajas y limitaciones de la elección	45
3.3.	Implementación: la solapa en Odoo	46
3.3.1.	Infraestructura preexistente del entorno CAETI/CIM .	46
3.3.2.	Stack tecnológico	46
3.3.3.	Materialización: mapeo de filtros y tubos al módulo Odoo	46
3.4.	Síntesis del capítulo	46
4.	Mapa de procesos	47
4.1.	Proceso general	47
4.2.	Recopilación de requerimientos	49
4.3.	Scrapping	50
4.4.	Definición de Contexto	54
4.5.	Procesamiento IAG	59
4.6.	Visualización previa	61
4.7.	Refinamiento iterativo	62
4.8.	Presentación de resultados	63
4.9.	Síntesis del capítulo	64
5.	Ejemplo de aplicación	65
5.1.	Selección del caso	65
5.2.	Configuración inicial	65
5.3.	Ejecución del scraping	65
5.4.	Definición de contexto y template utilizado	65
5.5.	Resultados del análisis generativo	65
5.6.	Discusión	65
6.	Conclusiones y Trabajo a Futuro	66

A. Tipos de datos del pipeline	67
A.1. Notación y criterios de diseño	67
A.2. Noticia	68
A.3. Embedding	69
A.4. Prompt	70
A.5. RespuestaIAG	71
A.6. Análisis	71
A.7. Veredicto	72
A.8. Resultado	73
A.9. Template	73
A.10. Los filtros como transformaciones de estado	74
A.11. La cadena de procedencia	75

Capítulo 1

Introducción

1.1. Motivación

El panorama informativo actual se caracteriza por la proliferación de portales digitales, que ofrecen una amplia variedad de noticias en línea. Hoy día, vivimos en un contexto de sociedades hipermediatizadas (Carlón, 2020), en donde el acceso y análisis de los discursos mediatizados a través de Internet presenta dos limitaciones fundamentales que deben ser sorteadas tanto por investigadores sociales, profesionales de la información como por gestores públicos para determinar y comprender su impacto en la sociedad.

1. La presencia de atributos sustanciales de los discursos (como metadatos descriptivos o técnicos) que residen en la “infraestructura subyacente” (Couldry and Hepp, 2017), siendo inaccesibles al “ojo desnudo”, es decir, mediante la observación directa.
2. La magnitud de los paquetes textuales y el volumen de los datos y metadatos asociados demandan análisis pormenorizados para generar conocimiento relevante, una tarea abrumadora sin las herramientas adecuadas.

Esta necesidad se inscribe en un contexto más amplio dentro de las ciencias sociales y humanas, marcado por lo que se ha denominado el “giro computacional” (Berry, 2011). Este giro exige la incorporación de herramientas, técnicas y métodos que permitan enriquecer y sofisticar los procedimientos de análisis a través de su computarización (Meunier, 2022). La relevancia

de este enfoque se extiende también a sectores de gestión gubernamental, donde la información actualizada proveniente de las conversaciones públicas es crucial para la toma de decisiones y el diseño de políticas públicas.

Para superar este desafío, se necesitan instrumentos de análisis de texto más sofisticados que permitan identificar correlaciones entre noticias, aco-
plamiento y cohesión de los términos, conceptos y significantes. Estos ins-
trumentos proporcionarán a los investigadores un sólido respaldo tecnológico
para automatizar y optimizar sus protocolos de estudio, incluyendo la for-
mación de la agenda mediática y la difusión de información de manera más
eficiente.

Una de las áreas de aplicación más relevantes de estos estudios se centra en
la influencia de los medios de comunicación tradicionales en la configuración
de la agenda pública, así como en el creciente impacto de las redes socia-
les como caja de resonancia de noticias y su llegada a públicos segmentados.
Desde la publicación del influyente artículo “La función de establecer la agen-
da de los medios de comunicación de masas” (McCombs and Shaw, 1972),
se ha reconocido el papel fundamental de los medios de comunicación en la
configuración de la percepción pública. No obstante, en la última década, la
irrupción de las redes sociales, primero con la blogosfera y posteriormente
con los sitios de aquellas, ha alterado significativamente este panorama.

El desarrollo de las mediciones y la obtención de indicadores referidos
a noticias publicadas en medios digitales se fue consolidando en los últimos
tiempos. Para este propósito se utilizan técnicas de análisis y **procesamien-
to de lenguaje natural** (NLP, por sus siglas en inglés de “Natural Language
Processing”); **la minería de texto** (Tan, 1999; Hotho et al., 2005); **ciencia
de datos** para la identificación de entidades, relaciones e indicadores claves
(KPI, por sus siglas en inglés de “Key Performance Indicator”); **clasifica-
ción mediante redes neuronales**, etc. Estas herramientas aunque útiles
para ciertas tareas, han demostrado limitaciones importantes para capturar
la complejidad de los “funcionamientos discursivos” que son esenciales pa-
ra el análisis social (Gindin and Busso, 2018). Estas limitaciones incluyen
dificultades para gestionar el contexto completo, mantener la atención en
partes relevantes del texto, identificar relaciones complejas entre conceptos
y adecuar la representación a tareas de análisis específicas, problemas que
a menudo surgían con algoritmos previos a las generaciones actuales de in-
teligencia artificial generativa (en adelante, IAG) basadas en arquitectura
transformer (Vaswani et al., 2017).

Por lo tanto, es necesario desarrollar nuevas estrategias de análisis que su-

peren estas restricciones, permitiendo acceder al nivel de los funcionamientos discursivos complejos y aplicando categorías y operaciones propias de diversas disciplinas científicas sociales.

Siguiendo con estos lineamientos, se plantea la hipótesis de la utilización de los servicios de IAG en un proceso original, combinados con técnicas de ciencias de datos y análisis de texto para organizar el análisis de noticias y representación de los resultados. La IAG se presenta como una tecnología prometedora, capaz de generar texto similar al humano y de capturar y reproducir relaciones complejas entre unidades lingüísticas, además de poder proveer cierto nivel de explicaciones sobre el significado de los patrones identificados en los textos analizados. Su aplicabilidad, calidad y confiabilidad se garantizará a través de un framework que involucre la supervisión, configuración, análisis y ecualizaciones por parte de expertos del dominio en diversas áreas de la investigación social.

Para su implementación se utilizarán los siguientes datos de entrada para la construcción de los “prompts” de la IAG a utilizar:

1. Los datos empíricos de las noticias con el agregado de información de contexto. Ejemplo: tipo de portal, georeferencia, importancia de la noticia, ubicación, etc.
2. Los objetivos requeridos para el análisis. Ejemplo: alcance, paradigma de análisis, referencias a tener en cuenta, etc.
3. El tipo de salida o representación del análisis. Ejemplo: narrativa, tablas, gráficos, etc.

1.2. Objetivos

Teniendo en cuenta las necesidades e hipótesis de solución se define un objetivo general para este trabajo de la siguiente manera:

Crear un framework para automatizar el análisis de noticias digitales a través de inteligencia artificial generativa.

Para la concreción del objetivo general propuesto se plantean los siguientes objetivos específicos.

OE1: Desarrollar una aplicación informática que permita gestionar el proceso integral de extracción y transformación de datos, análisis y muestra de resultados.

El alcance de este objetivo es crear un módulo específico que pueda ser integrado en la suite del framework denominado *Odoo Foundation*¹ en su versión comunitaria de código abierto. El módulo contará con servicios de gestión para los valores de parametrización de los algoritmos que intervienen en el proceso que se implementa. El usuario final podrá cargar, a través de formularios, la información necesaria para el acceso a los portales de noticias, las reglas de búsquedas de información y criterios sobre la aplicación de procedimientos. Se desarrollarán integraciones con otros módulos de la suite de Odoo que permitan la visualización de la información de resultados en tableros, informes y reportes adecuados para el propósito de aplicación.

OE2: Diseñar e implementar, en la aplicación informática, un proceso que organice las tareas y procedimientos de las etapas de recolección de datos, análisis y visualización.

En este objetivo específico, se pretende organizar y automatizar la hoja de ruta necesaria para satisfacer a los expertos del dominio y usuarios finales de la tecnología. Las tareas que se deben ejecutar forman parte de un proceso que debe ser diseñado e implementado tecnológicamente.

OE3: Diseñar e implementar los procedimientos de acceso a los servicios de IAG guiado por “prompts”.

Los servicios de inteligencia artificial generativa que se utilizarán en las tareas específicas del proceso, serán diseñados a través de un estilo de arquitectura que permita una adecuada integración al proceso mencionado. Para este propósito, se utilizarán los fundamentos del estilo de arquitectura denominada Tubos y Filtros (Garlan and Shaw, 1994; Shaw and Garlan, 1996).

¹https://www.odoo.com/es_ES

Capítulo 2

Análisis bibliográfico?

En el contexto de un ecosistema mediático caracterizado por la hipermedia-tización y la proliferación de datos, los métodos tradicionales de análisis de contenido enfrentan desafíos sin precedentes. Este capítulo aborda las bases teóricas y técnicas necesarias para fundamentar una infraestructura que automatice el análisis de noticias digitales. Se presenta primero el marco teórico que define el campo conceptual; luego se documenta la revisión sistemática de la literatura (RSL) que guió la selección de fuentes, junto con las preguntas de investigación, la metodología empleada y los estudios incluidos; finalmente se analiza el estado del arte en tres ejes temáticos, se sintetizan las brechas detectadas y se cierra con las implicancias para esta tesina.

2.1. Marco Teórico

Según Luhmann (2000, p. 1): “*Todo lo que sabemos acerca de nuestra sociedad, o más aún, acerca del mundo en el que vivimos, lo conocemos a través de los medios de comunicación masiva.*”¹ Esta afirmación no es una observación trivial sobre el flujo de información: Luhmann (2000) sostiene que los medios masivos constituyen un sistema social funcionalmente diferenciado que *produce* la realidad social en lugar de meramente reflejarla. Si lo que conocemos sobre el mundo nos llega mediado por instituciones cuyo criterio de selección obedece a su propia lógica operativa, entonces el estudio de esa mediación se vuelve política y científicamente ineludible.

¹Las traducciones de citas originalmente en otros idiomas son propias del autor, salvo indicación contraria.

La forma más influyente en que se ha estudiado empíricamente este fenómeno es la teoría del *agenda-setting*. McCombs and Shaw (1972) formularon la hipótesis de que “*los medios masivos establecen la agenda de cada campaña política, influyendo en la prominencia de las actitudes hacia los temas políticos*”. Recuperando una observación previa de Cohen, los autores sintetizaron este efecto en una fórmula que se volvió canónica: la prensa “*puede no ser exitosa la mayor parte del tiempo en decirle a la gente qué pensar, pero es asombrosamente exitosa en decirle a sus lectores sobre qué pensar*”. El estudio empírico de McCombs and Shaw (1972) sobre la elección presidencial estadounidense de 1968 en Chapel Hill encontró correlaciones cercanas a la unidad entre el énfasis temático de los medios y los temas que los votantes consideraban importantes, estableciendo así el primer nivel de la agenda-setting: la transferencia de prominencia (*salience*) desde la agenda mediática hacia la agenda pública.

Dos décadas más tarde, McCombs et al. (1997) extendieron la teoría al notar que los objetos sobre los que los medios construyen agenda –candidatos, temas, eventos– poseen *atributos*, tanto sustantivos como afectivos. Así, “*el primer nivel de la agenda-setting es la transmisión de la prominencia de los objetos; el segundo nivel de la agenda-setting es la transmisión de la prominencia de los atributos*” (McCombs et al., 1997, p. 704). Esta extensión vincula la agenda-setting con el concepto de *framing* desarrollado por Entman: enmarcar es seleccionar qué atributos de un objeto se vuelven prominentes en un texto comunicativo. La famosa fórmula de Cohen se reescribe en consecuencia: los medios no solo nos dicen “*sobre qué pensar, sino que también nos dicen cómo pensar al respecto*” (McCombs et al., 1997, p. 716). Este segundo nivel sustenta los estudios contemporáneos que miden computacionalmente el sesgo y el sentimiento en la cobertura mediática, analizados más adelante en la Sección 2.3.

El paisaje en el que opera la agenda-setting cambió radicalmente desde McCombs y Shaw. Couldry and Hepp (2017) caracterizan la situación contemporánea como *deep mediatization*, un régimen marcado por tres procesos simultáneos: la *datafication* (cada encuentro mediado deja un rastro de datos), la fragmentación profunda de las audiencias y la incorporación de plataformas y algoritmos a la construcción misma del acontecimiento mediático. En palabras de los autores, “*las construcciones algorítmicas de la realidad pasan a formar parte de la construcción de los eventos mediáticos*” (Couldry and Hepp, 2017, p. 3), transformación que obliga a analizar los procesos mediáticos contemplando el rol de datos y algoritmos. En sintonía

con esta caracterización, Carlón (2020) sitúa el concepto de *sociedad hiper-mediatizada* como el contexto general en el que esta tesina inscribe su objeto de estudio.

Esta transformación del objeto exige una transformación correlativa de los métodos. Berry (2011) identifica este desplazamiento como un *computational turn*: la creciente dependencia de las ciencias sociales y humanidades respecto de técnicas computacionales para acceder a sus objetos. Berry advierte que las técnicas computacionales “*no son meramente un instrumento empuñado por los métodos tradicionales; más bien, tienen efectos profundos sobre todos los aspectos de las disciplinas*” (Berry, 2011, p. 13): introducen nuevos modos de análisis –como el *distant reading* de Moretti recuperado por Berry (2011), según el cual los corpus literarios masivos no pueden comprenderse sumando lecturas individuales sino tratándolos como sistemas colectivos– y reconfiguran las preguntas que las disciplinas pueden formular. Meunier (2022) extiende esta línea hacia la semiótica computacional, ofreciendo un sustrato teórico complementario para la articulación entre código, signos y significado.

El giro computacional aplicado al análisis del lenguaje natural cuenta con un punto de inflexión técnico preciso: la arquitectura *Transformer* propuesta por Vaswani et al. (2017). A diferencia de las redes recurrentes y convolucionales previas, el Transformer se basa exclusivamente en mecanismos de *self-attention*, entendidos como una función que relaciona distintas posiciones dentro de una secuencia para construir una representación contextual de la misma. La entrada y la salida del modelo se construyen sobre *embeddings*: representaciones vectoriales aprendidas que codifican unidades lingüísticas en un espacio continuo de alta dimensionalidad. Sobre esta arquitectura se construyen los modelos generativos contemporáneos –LaMDA, GPT-3, GPT-4– capaces de traducir, clasificar y generar respuestas analíticas complejas; así como los modelos multimodales –DALL-E 2 y sucesores– útiles para complementar representaciones visuales del discurso.

Estos desarrollos teóricos y técnicos encuentran un antecedente empírico relevante en el trabajo de tesis doctoral de Pablo Rey-Mazón, “Color of Corruption” (Rey-Mazón, 2023), que articula varias herramientas y una metodología novedosas para medir el proceso de *agenda-setting* en un ecosistema mediático complejo. Según Rey-Mazón, la metodología tradicional de análisis de contenido se basa en el supuesto de que el contenido ya está completo y estático para su estudio, lo que ignora el flujo constante de noticias y su influencia sobre la recepción. En contraste, su software **HomepageX** rastrea

la posición de los elementos noticiosos en las páginas de inicio durante varios días, permitiendo reconstruir los flujos y ciclos de noticias en los sitios web mediante el cruce entre una base de datos construida por scraping horario y análisis automatizado de contenido. La herramienta complementaria **Color of Corruption** consiste en una base de datos de imágenes de noticias sobre corrupción etiquetada por tono emocional, enfoque narrativo y presencia de actores específicos, soporte de la metodología denominada *Visual Agenda-Setting Analysis*. La cercanía conceptual y metodológica del trabajo de Rey-Mazón con la propuesta de esta tesina lo vuelve una referencia central.

Disponer de modelos generativos no alcanza, sin embargo, para construir una infraestructura utilizable: es necesario orquestar las tecnologías mediante estándares de procesos de negocio. Trabajos como el de Prades et al. (2013) establecen metodologías para implementar flujos de trabajo en BPMN de acuerdo con el estándar ANSI/ISA-95. Investigaciones recientes, como la de Bertagnolio et al. (2023), demuestran la viabilidad de incorporar modelos generativos dentro de mapas de procesos tecnológicos automatizados. La articulación entre marcos teóricos, instrumentos computacionales y orquestación procedimental que se propone en esta tesina se inscribe en esa misma línea.

2.2. Revisión Sistemática de la Literatura

Sobre la base del marco teórico anterior, se llevó a cabo una revisión sistemática de la literatura (RSL) orientada a identificar trabajos empíricos y de ingeniería que combinen análisis de noticias, web scraping, embeddings y modelos de lenguaje extenso. Esta sección presenta las preguntas que guiaron la revisión, la metodología de búsqueda y selección, y el conjunto final de estudios incluidos.

2.2.1. Preguntas de Investigación

La revisión se orientó a responder las siguientes preguntas:

PR1: ¿Cuáles son las limitaciones de las técnicas tradicionales de Procesamiento de Lenguaje Natural (NLP) frente a la complejidad del análisis discursivo en medios digitales?

- PR2: ¿Qué metodologías y herramientas informáticas se están utilizando para evaluar el agenda-setting y el impacto mediático en la actualidad?
- PR3: ¿Cuáles son las brechas estructurales y procedimentales en los pipelines de IAG actuales al integrar web scraping y embeddings para la investigación social?

2.2.2. Metodología de Búsqueda y Selección

La presente revisión se condujo siguiendo las directrices del protocolo PRISMA 2020 (Page et al., 2021), con el propósito de identificar estudios que combinaran análisis de noticias, web scraping, embeddings y modelos de lenguaje extenso (LLM).

La recolección bibliográfica se ejecutó durante mayo de 2026 sobre seis bases de datos académicas: SciSpace (búsqueda básica), SciSpace con filtro de texto completo, SciSpace Library, biblioteca personal del autor, Google Scholar y ArXiv. La cadena de búsqueda exacta aplicada se detalla en la Figura 2.1. Los conteos por base de datos fueron: SciSpace (n=100), SciSpace texto completo (n=100), SciSpace Library (n=8), biblioteca personal (n=6), Google Scholar (n=20) y ArXiv (n=20), totalizando 254 registros identificados.

Los **criterios de inclusión** aplicados fueron:

- CI1: Artículos, tesis o actas de conferencias.
- CI2: Publicados entre 2020 y 2025.
- CI3: Escritos en español o inglés.
- CI4: Presentan detalles técnicos sobre la arquitectura de software propuesta.
- CI5: Combinan al menos dos de las áreas centrales de la revisión: web scraping, embeddings, LLMs/IAG, agenda mediática.

Complementariamente, los **criterios de exclusión** aplicados fueron:

- CE1: Trabajos puramente teóricos sin componente de software.
- CE2: Basados en tecnologías pre-transformer u obsoletas.
- CE3: Metodología no replicable o sin descripción técnica suficiente.

CE4: Fuera del dominio de análisis de medios digitales o noticias.

CE5: Texto completo no disponible públicamente.

El proceso de selección se desarrolló en cuatro fases ejecutadas por el autor. Primero se eliminaron los duplicados entre bases ($n=20$), reduciendo el corpus a 234 registros. A continuación se realizó un cribado por título y resumen aplicando los criterios de inclusión, excluyendo 130 registros por baja relevancia y conservando 104. Sobre estos se buscó recuperar el texto completo, descartando 54 informes por no contar con acceso público al documento (CE5). Finalmente, los 50 informes recuperados fueron evaluados a texto completo aplicando los criterios de exclusión, descartando 41: 20 por estar fuera del tema principal (CE4), 12 por carecer de metodología replicable (CE3) y 9 por idioma no admitido (CI3 incumplido). El conjunto final quedó conformado por 9 estudios, presentados en la Sección 2.2.3 y analizados en la Sección 2.3.

Diagrama de Selección Bibliográfica — Protocolo PRISMA



Figura 2.1: Diagrama de selección bibliográfica según el protocolo PRISMA (Page et al., 2021).

2.2.3. Estudios Incluidos

El proceso de selección descrito en la sección anterior arrojó nueve estudios incluidos en la revisión, distribuidos de manera desigual entre los tres ejes temáticos: dos trabajos abordan las limitaciones del NLP tradicional (PR1), dos las aplicaciones computacionales del *agenda-setting* (PR2) y cinco las arquitecturas de software con IAG para el procesamiento de noticias (PR3). Este desbalance refleja la madurez relativa de cada línea: mientras que las dos primeras presentan estudios más bien fundacionales o de validación metodológica, la tercera concentra la mayor parte de la producción reciente, donde proliferan prototipos y sistemas aplicados que combinan extracción, vectorización e inferencia.

Una lectura transversal del corpus revela un patrón arquitectónico recurrente: la mayoría de los trabajos articula una cadena de procesamiento que va de la ingesta de contenido –ya sea por *scraping* directo o por consumo de APIs noticiosas– a la representación semántica mediante *embeddings* o esquemas RAG, y de allí a la inferencia con modelos de lenguaje extenso (Neupane et al., 2025; Behbahanian, s.f.; Somasundharam et al., 2025; Ameer, 2025). Las variantes no afectan la estructura general sino los énfasis: algunos priorizan deduplicación y agregación (Behbahanian, s.f.), otros la entrega multimedia o personalizada (Ameer, 2025; Begum J and D, 2024), y otros la trazabilidad de fuentes (Somasundharam et al., 2025). Esta convergencia técnica confirma que el patrón existe como herencia disponible, pero también vuelve más visibles las dimensiones que ningún estudio resuelve por completo.

En particular, las limitaciones que reportan los propios autores conforman un mapa de zonas no cubiertas que se repiten a lo largo del corpus: la ausencia de validación con usuarios o expertos (Behbahanian, s.f.; Neupane et al., 2025), el sacrificio de la profundidad interpretativa en favor de la distribución o la *performance* (Ameer, 2025; Somasundharam et al., 2025), la dependencia de muestras acotadas o casos únicos (Pavlyshenko, 2024; Samuel et al., 2025) y la falta de estandarización cross-plataforma (Haider et al., 2025). La Tabla 2.1 sintetiza este panorama agrupado por pregunta de investigación; la columna “Estudio” incorpora entre paréntesis el sistema o stack distintivo de cada trabajo.

Tabla 2.1: Estudios incluidos en la revisión sistemática, agrupados por pregunta de investigación.

Estudio	Aporte central	Limitación
<i>PR1 — Limitaciones del NLP tradicional frente al discurso mediático</i>		
Milbauer et al. (2023) (<i>NewsSense</i>)	Evidencia el límite del NLP clásico para sintetizar perspectivas contradictorias y verificar sin referencias externas	Foco en <i>sensemaking</i> del lector; no escala a corpus masivos
Pavlyshenko (2024) (RAG GPT-4o + bayesiano)	La IAG supera al NLP estadístico en análisis cualitativo, pero requiere modelado bayesiano para cuantificar la incertidumbre	Muestra acotada; caso único (elecciones US 2024)
<i>PR2 — Aplicaciones computacionales del agenda-setting de segundo nivel</i>		
Haider et al. (2025) (<i>Media Bias Detector</i>)	Medición sistemática de selección y encuadre mediático a escala	Sistema en curso; falta estandarización cross-plataforma de etiquetas
Samuel et al. (2025) (ML + LLMs sobre corpus)	Traza dinámicas emocionales (<i>AI phobia</i>) propagadas por la cobertura mediática	Tema único; corpus específico
<i>PR3 — Arquitecturas de software con IAG para el procesamiento de noticias</i>		
Behbahanian (s.f.) (<i>GlimzAI</i>)	Deduplicación semántica y agregación end-to-end de noticias	Sin validación con usuarios; foco en distribución
Neupane et al. (2025) (Node.js + GPT-4o)	Clustering temático y generación automática de contenido para portales autónomos	Sin validación con expertos
Ameer (2025) (<i>NexGen MediaCraft</i>)	Broadcasting de noticias en tiempo real con presentador virtual	Foco en presentación; no aborda análisis interpretativo
Begum J and D (2024) (GPT-3.5 + BERTweet)	Reportes diarios personalizados con resumen en dos niveles y análisis de sentimiento	Foco en personalización individual
Somasundharam et al. (2025) (LangChain + LLaMA-3)	RAG con trazabilidad de fuentes y baja latencia para consultas multi-URL	Foco en performance; no aborda análisis interpretativo

2.3. Estado del Arte

Sobre la base del Marco Teórico (Sección 2.1), esta sección analiza estudio por estudio cómo el corpus incluido aborda cada una de las preguntas de investigación. Las subsecciones siguen el orden de las tres PR.

2.3.1. Limitaciones del Procesamiento de Lenguaje Natural (NLP) Tradicional

Esta subsección aborda la PR1 examinando las limitaciones que las técnicas tradicionales de NLP enfrentan al analizar el discurso mediático contemporáneo. El análisis de grandes volúmenes de noticias ha dependido históricamente de técnicas de NLP centradas en la detección de palabras clave y análisis de sentimiento superficial. Sin embargo, autores como Milbauer et al. (2023) sostienen que estas técnicas fallan al capturar el *sensemaking* cross-documental, es decir, la capacidad de contrastar y sintetizar perspectivas contradictorias presentes en múltiples artículos. Mientras que el NLP tradicional trata los documentos como entidades aisladas, la necesidad actual de investigadores sociales exige interfaces que evalúen el apoyo o contradicción de la evidencia en tiempo real (Milbauer et al., 2023).

Por otro lado, Pavlyshenko (2024) evalúa el uso de modelos GPT frente a métodos estadísticos tradicionales, concluyendo que, aunque la IAG ofrece una capacidad interpretativa superior para el análisis cualitativo, aún presenta incertidumbres significativas en la cuantificación de datos. Esta postura contrasta con enfoques puramente algorítmicos al sugerir que la IAG no reemplaza al NLP estadístico, sino que requiere de marcos complementarios (como la regresión bayesiana) para interpretar los sesgos inherentes en las salidas generadas (Pavlyshenko, 2024).

2.3.2. Aplicaciones Computacionales del Agenda-Setting de Segundo Nivel

Esta subsección aborda la PR2 examinando cómo trabajos recientes han operacionalizado computacionalmente el agenda-setting de segundo nivel propuesto por McCombs et al. (1997), particularmente en lo relativo a dos atributos centrales: el sesgo y el encuadre (*framing*) por un lado, y el sentimiento por otro. Haider et al. (2025) proponen un framework para detectar el sesgo

mediático a escala, subrayando que la selección y el encuadre pueden medirse sistemáticamente mediante pipelines de scraping integrados con LLMs. Este enfoque permite una vigilancia continua de la responsabilidad mediática, algo que los métodos manuales de codificación no podían alcanzar por limitaciones de escala (Haider et al., 2025).

Complementariamente, el estudio de Samuel et al. (2025) sobre la difusión del miedo hacia la inteligencia artificial evidencia cómo la cobertura mediática amplifica sentimientos públicos. Al emplear una combinación de aprendizaje automático y LLMs, los autores logran trazar dinámicas emocionales en el corpus noticioso, demostrando que la infraestructura técnica es capaz de revelar no solo qué temas están en la agenda, sino qué emociones colectivas se están propagando a través de ella (Samuel et al., 2025).

2.3.3. Arquitecturas de Software con IAG para el Procesamiento de Noticias

Esta subsección aborda la PR3 examinando las arquitecturas de software emergentes que integran *web scraping*, *embeddings* y modelos generativos en *pipelines* reproducibles para sistematizar el procesamiento y análisis de noticias. La implementación de infraestructuras técnicas ha evolucionado hacia sistemas modulares y agentes autónomos. Behbahanian (s.f.) describe el sistema *GlimzAI*, que utiliza flujos de trabajo “agentic” y *embeddings* de Sentence-BERT para la deduplicación semántica. Este enfoque es fundamental para el análisis de noticias, ya que permite gestionar la redundancia informativa de múltiples portales.

Asimismo, Neupane et al. (2025) demuestra la eficacia de combinar un backend de scraping basado en Node.js con modelos como GPT-4o para orquestar pipelines que van desde la extracción y clustering semántico hasta la publicación automatizada de contenido (Neupane et al., 2025).

Variantes contemporáneas amplían este patrón arquitectónico en distintas direcciones. Ameer (2025) propone *NexGen MediaCraft*, un sistema que combina *scraping* horario, base vectorial e inferencia con un modelo *DeepSeek* ajustado para producir noticias en tiempo real entregadas mediante un presentador virtual 3D. Begum J and D (2024) integra GoogleNews y Newspaper4k para extracción, GPT-3.5-turbo-16k para resumen en dos niveles y BERTweet para análisis de sentimiento, distribuyendo reportes personalizados por correo electrónico. Por su parte, Somasundharam et al. (2025) aplica

una arquitectura RAG con FAISS, *embeddings* de Hugging Face y LLaMA-3 sobre LangChain, priorizando la trazabilidad de las fuentes y la baja latencia en respuestas a consultas multi-URL. En conjunto, estos trabajos confirman la consolidación de un patrón común –scraping, indexación vectorial e inferencia con LLMs– pero exhiben divergencias significativas en sus objetivos finales (entrega multimedia, reporte personalizado, recuperación trazable) y en los criterios de evaluación aplicados.

2.4. Síntesis Crítica y Brechas en la Investigación

Al contrastar las fuentes, se observa una convergencia clara en el uso de *embeddings* para la representación semántica del contenido y pipelines modulares de extracción, procesamiento y publicación (Neupane et al., 2025; Behbahanian, s.f.). No obstante, surgen divergencias en cuanto a la prioridad: mientras algunos autores se enfocan en la eficiencia de la entrega de contenidos (Behbahanian, s.f.), otros priorizan la verificabilidad y la explicabilidad del rastro documental (Milbauer et al., 2023).

Se han identificado las siguientes brechas que el framework propuesto en esta tesina busca llenar:

- **Escalabilidad y consistencia de anotaciones:** Aunque se busca medir la selección y el encuadre (agenda-setting), falta un estándar interoperable de etiquetas (tópico, tono, tipo de cobertura, tal como lo describe Haider et al. (2025)) y formas de evaluar su fiabilidad cross-plataforma.
- **Robustez del Scraping:** La mayoría de los prototipos documentados no abordan estrategias de mantenimiento ante cambios estructurales en portales dinámicos (Haider et al., 2025).
- **Trazabilidad en Ciencias Sociales:** Existe poca evaluación sobre la fidelidad de la evidencia extraída. Hacen falta mecanismos uniformes que integren los *embeddings* con los metadatos del scraping para auditar (y justificar ante el investigador social) qué fragmentos apoyan cada afirmación generada.

- **Calibración de Incertidumbre:** Pocos frameworks integran la cuantificación de incertidumbre en los scores de análisis de sentimiento o tópicos, un aspecto crítico destacado por Pavlyshenko (2024).
- **Gobernanza de publicación y responsabilidad editorial:** Existen riesgos por la publicación automática de análisis sin filtros de veracidad o atribución, siendo críticos los procedimientos *human-in-the-loop* ante controversias.

Propuesta de valor: Ante estas brechas, un framework modular integrado proporcionará scraping resistente, indexación por embeddings, y control sobre la incertidumbre, mitigando los riesgos de la automatización ciega y aumentando la reproducibilidad y la trazabilidad para los investigadores.

2.5. Conclusión del Capítulo

La revisión sistemática presentada en este capítulo permite delimitar con precisión el punto de partida de esta tesina, tanto en lo que la literatura ya ha consolidado como en lo que aún permanece sin resolver.

En el plano de lo consolidado, los nueve estudios analizados confirman que el patrón arquitectónico que combina *web scraping*, indexación mediante *embeddings* e inferencia con modelos de lenguaje extenso es técnicamente viable y reproducible en distintos dominios de aplicación (Behbahanian, s.f.; Neupane et al., 2025; Somasundharam et al., 2025). Asimismo, queda establecido que la inteligencia artificial generativa supera al NLP estadístico tradicional en tareas interpretativas (Milbauer et al., 2023; Pavlyshenko, 2024) y que es posible operacionalizar computacionalmente el *agenda-setting* de segundo nivel a escala (Haider et al., 2025; Samuel et al., 2025). Esta base no constituye, por lo tanto, un riesgo a explorar en este trabajo, sino el suelo técnico sobre el que se construye.

En el plano de lo no resuelto, la Síntesis Crítica (Sección 2.4) identificó cinco brechas que los sistemas existentes no abordan de manera integral: la ausencia de estándares interoperables para anotar tópico, tono y encuadre de forma comparable entre plataformas; la fragilidad del *scraping* ante cambios estructurales de los portales; la falta de trazabilidad documental entre la evidencia extraída y las afirmaciones generadas por los modelos; la escasa calibración de la incertidumbre en los *scores* de análisis; y la carencia de procedimientos de gobernanza editorial *human-in-the-loop*. Estas brechas

configuran el espacio problemático específico desde el cual arranca esta tesis.

A partir de ese punto de partida, los tres objetivos específicos definidos en el Capítulo 1 responden a brechas concretas detectadas en la revisión. El OE1 (módulo Odoos) institucionaliza el flujo extracción-análisis-visualización como aplicación gestionable y auditable, atendiendo a las carencias de gobernanza y trazabilidad. El OE2 (mapa de procesos) formaliza la hoja de ruta procedimental que dota de reproducibilidad y robustez a las etapas de recolección y análisis. El OE3 (procedimientos de acceso a IAG mediante *prompt engineering* bajo arquitectura Tubos y Filtros) busca consistencia de anotaciones y trazabilidad de inferencias en la capa generativa. En conjunto, el framework propuesto no aspira a reemplazar el patrón arquitectónico consolidado por la literatura, sino a completarlo en las dimensiones procedimentales y de rigor científico que los prototipos relevados aún no resuelven.

El capítulo siguiente desarrolla la primera capa de esta propuesta: la aplicación de *scraping* que materializa el componente de extracción del framework.

Capítulo 3

Arquitectura

Este capítulo presenta la arquitectura de la infraestructura propuesta. A diferencia del Capítulo 4, donde los procesos se describen de manera agnóstica a la implementación, aquí se aborda la dimensión técnica: las decisiones de estilo arquitectónico, los componentes que materializan cada capa del sistema y el stack tecnológico sobre el que se construye.

La propuesta no se concibe en el vacío. Esta tesina se inscribe en una línea de investigación interdisciplinaria desarrollada por el Centro de Altos Estudios en Tecnología Informática (CAETI) de la Universidad Abierta Interamericana en articulación con el Centro de Investigaciones de Medios (CIM) de la Universidad Nacional de Rosario, en cuyo marco se ha venido construyendo una infraestructura técnica para la extracción y análisis de noticias de portales digitales mediante inteligencia artificial generativa. Esa infraestructura preexistente provee el sustrato sobre el cual esta tesina opera: una plataforma Odoo Community con módulos para la configuración y monitoreo de portales digitales, una herramienta de scraping basada en spiders parametrizables que combina librerías como BeautifulSoup, Selenium y Newspaper3k, y una capa inicial de procesamiento mediante *retrieval-augmented generation* (RAG) y modelos de lenguaje de gran escala.

Sobre esa base, el aporte original de la presente tesina se concentra en tres artefactos técnicos. En primer lugar, un módulo *addon* específico para Odoo que integra y unifica las capacidades necesarias para automatizar el ciclo completo de análisis, satisfaciendo el objetivo específico OE1. En segundo lugar, un panel de creación de *templates* de *prompt* que reifica como subsistema gestionado lo que en aplicaciones convencionales basadas en modelos de lenguaje suele resolverse de manera *ad hoc*, habilitando la configuración,

parametrización y reutilización de plantillas por parte de expertos del dominio. En tercer lugar, un pipeline de procesamiento orientado a inteligencia artificial generativa diseñado bajo el estilo arquitectónico de Tubos y Filtros (Garlan and Shaw, 1994; Shaw and Garlan, 1996), que organiza la transformación progresiva del input —desde los datos empíricos extraídos hasta el resultado analítico final— como una composición de filtros desacoplados con formatos de mensaje explícitos entre etapas, en línea con el objetivo específico OE3.

La elección de Tubos y Filtros como estilo articulador del procesamiento generativo no es arbitraria. Refleja, por una parte, una tendencia documentada en la literatura sobre la integración de modelos generativos dentro de mapas de procesos tecnológicos automatizados (Bertagnolio et al., 2023), donde la modularización del flujo se ha consolidado como condición para la mantenibilidad, la auditabilidad y la sustituibilidad de los componentes algorítmicos. Por otra parte, atiende de manera específica a brechas identificadas en la revisión sistemática del Capítulo 2, según se desarrolla en la Sección 3.1.3.

El capítulo se organiza en cuatro secciones. La Sección 3.1 presenta el estilo de Tubos y Filtros: sus componentes, conectores, invariantes y modelo computacional, y justifica su elección como estilo articulador del procesamiento generativo de noticias. La Sección 3.2 aplica ese estilo al caso concreto de la presente tesina, caracterizando el framework como un sistema de Tubos y Filtros cuya fuente son los portales digitales de noticias y cuyo sumidero es la capa de presentación de resultados, y siguiendo el esquema descriptivo del catálogo de Cristiá and Pomponio (2025). La Sección 3.3 describe la implementación concreta del framework como un módulo *addon* de Odoo, distinguiendo la infraestructura preexistente del entorno institucional de los componentes desarrollados específicamente para esta tesina. Finalmente, la Sección 3.4 sintetiza el capítulo y anticipa cómo los elementos arquitectónicos aquí descriptos se operativizan procesualmente en el Capítulo 4.

3.1. El estilo Tubos y Filtros

3.1.1. Componentes, conectores y patrón estructural

El estilo de Tubos y Filtros define dos clases de elementos. Los *filtros* son los componentes activos del sistema: unidades de procesamiento independientes

que reciben datos por uno o más *puertos* de entrada y producen datos sobre uno o más *puertos* de salida (Cristiá and Pomponio, 2025). Cada filtro encapsula una transformación local sobre el flujo que recibe, sin compartir estado con otros filtros y sin conocer su identidad. Los *tubos* son los conectores: canales unidireccionales que transportan datos del puerto de salida de un filtro al puerto de entrada de otro, preservando el orden de los elementos transmitidos y sin modificar su contenido (Garlan and Shaw, 1994; Shaw and Garlan, 1996).

La configuración estructural canónica del estilo se organiza como un grafo dirigido acíclico que va desde una o más *fuentes* hasta uno o más *sumideros*. Las fuentes son los componentes externos al sistema que alimentan datos al pipeline —típicamente entradas estándar, archivos, dispositivos o servicios externos—; los sumideros son los componentes externos que reciben los datos emitidos por el sistema. El flujo de información se propaga progresivamente entre filtros adyacentes a través de los tubos, sin retroceso ni ciclos en su forma canónica (Cristiá and Pomponio, 2025).

Una propiedad estructural distintiva del estilo es su *cierre composicional*: un sistema entero de Tubos y Filtros constituye en sí mismo un filtro, en la medida en que admite ser tratado como una unidad de procesamiento con sus propios puertos de entrada y salida (Cristiá and Pomponio, 2025, p. 24). Esta propiedad habilita el diseño jerárquico —en el que un filtro puede a su vez descomponerse internamente en una subred de filtros— y formaliza la noción de que el sistema completo es una composición funcional de los filtros que lo integran (Garlan and Shaw, 1994). Esta lectura composicional reaparece de forma operativa en las bibliotecas modernas de procesamiento de datos, en las que un *pipeline* completo constituye en sí mismo una unidad reutilizable equiparable a sus componentes (Buitinck et al., 2013). La descripción detallada de los componentes específicos del framework propuesto, así como de las fuentes y sumideros concretos sobre los que opera, se presenta en la Sección 3.2.

3.1.2. Modelo computacional e invariantes

Cada filtro de un sistema de Tubos y Filtros opera de manera independiente: no comparte memoria con los demás filtros y procesa los datos disponibles en sus puertos de entrada sin coordinación explícita con el resto del sistema. La literatura distingue dos modalidades de filtros según cómo accedan al flujo entrante (Cristiá and Pomponio, 2025). Los filtros *activos* revisan periódicamente

camente sus puertos de entrada en busca de datos disponibles y, cuando los detectan, los consumen y producen la salida correspondiente sobre sus puertos de salida; los filtros *pasivos* exponen una interfaz que es invocada por los tubos para comunicarle o solicitarle datos. La elección entre una y otra modalidad responde a consideraciones de implementación —asociadas, por ejemplo, al uso de hilos de ejecución, a la sincronización entre productores y consumidores y al manejo de buffers limitados— y no altera la semántica del estilo.

El estilo se define formalmente por cuatro *invariantes esenciales* que cualquier diseño que se reclame como Tubos y Filtros debe verificar (Cristiá and Pomponio, 2025, p. 23); las tres primeras formalizan las invariantes que Garlan and Shaw (1994) establecen para el estilo:

1. Ningún filtro conoce la identidad ni la existencia de los demás filtros del sistema.
2. Ningún filtro depende del estado interno de los demás filtros, ni puede observarlo o modificarlo.
3. La corrección de la salida producida por el sistema no depende del orden en que cada filtro realice sus cálculos individuales.
4. Los tubos transmiten los datos entre filtros sin alterar su contenido.

Estas invariantes constituyen, en conjunto, una condición de *independencia funcional* entre los componentes del sistema. Tres consecuencias arquitectónicas se desprenden directamente de ellas. En primer lugar, la *sustituibilidad* de cada filtro: cualquier filtro puede ser reemplazado por otro que respete su interfaz de puertos y los tipos de datos transmitidos, sin necesidad de modificar el resto del sistema (Garlan and Shaw, 1994). En segundo lugar, la *auditabilidad lineal* del flujo: las transformaciones que sufren los datos al atravesar el sistema pueden inspeccionarse filtro por filtro, sin reconstruir interacciones implícitas. En tercer lugar, la *posibilidad de ejecución concurrente*: en la medida en que el orden de cómputo no incide sobre la corrección del resultado, los filtros pueden ejecutarse en paralelo o distribuirse entre máquinas distintas sin afectar la semántica del estilo. Conviene precisar que el invariante 3 refiere al orden de *scheduling* interno con que cada filtro realiza sus cálculos individuales, no a las dependencias causales que la topología del pipeline establece entre ellos: un filtro no puede consumir datos que aún no

fueron emitidos en el puerto al que está conectado, y esa restricción —propia del grafo de tubos— es una dependencia de flujo, no una dependencia de orden de cómputo en el sentido del invariante. La verificación de cómo se satisfacen estas condiciones en el diseño propuesto se aborda en la Subsección 3.2.7; el análisis de cuál de las propiedades arquitectónicas resulta más relevante para el caso de la presente tesina se discute en la Subsección 3.1.3.

3.1.3. Por qué Tubos y Filtros para el procesamiento generativo de noticias

La elección de un estilo arquitectónico para el procesamiento generativo de noticias debe responder a las necesidades específicas del problema, tal como se las delimitó en los capítulos previos. La revisión sistemática presentada en el Capítulo 2 identificó cinco brechas que los sistemas existentes no abordan de manera integral; de ellas, dos caen dentro del alcance de las decisiones de estilo arquitectónico y orientan la elección que se adopta en esta tesina: la *fragilidad del scraping* ante cambios estructurales de los portales, que demanda poder modificar o reemplazar la etapa de extracción sin reescribir el resto del sistema; y la *falta de trazabilidad documental* entre la evidencia extraída y las afirmaciones generadas por los modelos, que exige que cada transformación de los datos resulte localizable e inspeccionable. Las restantes —ausencia de estándares interoperables de anotación, escasa calibración de la incertidumbre y carencia de procedimientos de gobernanza editorial *human-in-the-loop*— exceden el ámbito de la decisión arquitectónica y se atienden en otros niveles de la propuesta: en el modelo de datos del módulo y en el mapa de procesos del Capítulo 4. A estas dos brechas se suman dos requisitos adicionales derivados del ecosistema tecnológico en el que la propuesta opera: la necesidad de poder sustituir los modelos generativos accesibles por API ante una evolución acelerada del mercado, y la necesidad de recombinar las etapas de procesamiento para acomodar distintos casos de estudio sin reescribir la infraestructura.

El estilo de Tubos y Filtros se ajusta a este perfil de problema. El catálogo de Cristiá and Pomponio (2025, p. 21) identifica entre los dominios canónicos de aplicación del estilo el procesamiento de cadenas, el procesamiento de señales y, en general, los sistemas cuyo flujo de información se conciba como una secuencia de transformaciones sobre un *stream* de datos. El análisis automatizado de noticias mediante modelos generativos, en tanto

procesamiento progresivo de texto que se inicia en la extracción y culmina en una representación analítica enriquecida, encuadra naturalmente en esta familia. Garlan and Shaw (1994) mencionan, además, los sistemas de transformación de información y los entornos distribuidos como dominios donde el estilo ha resultado históricamente efectivo.

La propiedad de *sustituibilidad* de los filtros, derivada de las invariantes 1 y 2 del estilo, atiende de manera conjunta a la brecha de fragilidad del scraping y al requisito de sustitución de modelos generativos. En la medida en que ningún filtro conoce la identidad de los demás ni accede a su estado interno, la etapa de extracción puede actualizarse —para adaptarse a un rediseño del portal monitoreado— sin afectar a las etapas posteriores; del mismo modo, el filtro responsable de invocar al modelo generativo puede ser reemplazado por otro que utilice un proveedor distinto, o un modelo distinto del mismo proveedor, sin necesidad de reescribir el resto del pipeline. La misma propiedad habilita la sustitución del filtro de enriquecimiento semántico —reemplazando, por ejemplo, una técnica de *embedding* por otra— sin alterar las etapas siguientes. Esta capacidad de evolución localizada resulta particularmente valiosa frente a infraestructuras cuyas dependencias externas evolucionan con frecuencia.

La *auditabilidad lineal* del flujo, segunda consecuencia derivada de las invariantes del estilo, responde directamente a la brecha de trazabilidad documental identificada en la revisión: la dificultad de reconstruir el camino que sigue la evidencia desde su extracción hasta las afirmaciones que el modelo generativo produce sobre ella. Cada transformación que los datos sufren al atravesar el sistema queda localizada en un filtro identificable, con una entrada y una salida definidas y sin interacciones implícitas mediadas por estados compartidos. Para una investigación que pretenda fundamentar afirmaciones sobre la cobertura mediática o el sesgo discursivo en términos verificables, esta propiedad no es accesorio: constituye una condición de posibilidad para la revisión metodológica y para la reproducibilidad del análisis. La separación explícita entre las etapas de extracción, enriquecimiento, composición de contexto e invocación generativa permite, además, distinguir las decisiones técnicas que afectan al corpus de aquellas que afectan a la interpretación, dimensión sobre la que el experto del dominio debe poder ejercer un control directo.

Por último, el *cierre composicional* del estilo —el hecho de que un sistema de Tubos y Filtros sea en sí mismo un filtro— y la modularidad consiguiente atienden al requisito de diversidad de los casos analíticos. La caracterización

del problema como una composición funcional de filtros habilita la construcción de pipelines especializados para distintos casos de estudio, en los que las etapas de procesamiento pueden recombinarse, agregarse o suprimirse en función del objetivo concreto del experto del dominio, sin alterar las garantías estructurales del estilo (Hohpe and Woolf, 2003). La adopción de Tubos y Filtros es coherente, además, con la forma en que los sistemas contemporáneos de aprendizaje automático y de modelos generativos se construyen por composición de componentes modulares: bibliotecas como *scikit-learn* organizan el flujo como una composición de transformaciones que constituye en sí misma una unidad reutilizable (Buitinck et al., 2013), y arquitecturas como la generación aumentada por recuperación combinan un modelo generativo con un componente de recuperación en un *pipeline* entrenado de extremo a extremo (Lewis et al., 2020).

Conviene anticipar que el diseño que se adopta no constituye una instancia pura del estilo, sino una variante *heterogénea* que combina la red de tubos y filtros con un repositorio compartido. Cada uno de los dos ingredientes de esta variante cuenta con respaldo independiente en la literatura: la admisión de un repositorio compartido junto a la red de tubos configura una *arquitectura heterogénea* en el sentido de Garlan and Shaw (1994), mientras que el mecanismo de transportar por el conector una referencia al contenido —y no la carga útil— corresponde al patrón *Claim Check* de Hohpe and Woolf (2003), tratado además como patrón de integración de primer orden en la literatura revisada por pares (Ritter, 2014). La combinación específica de ambos —tubos que transportan *claim checks* mientras los datos residen en un almacén separado— no constituye, hasta donde alcanza esta revisión, un patrón consolidado en la literatura académica, sino una síntesis propia de este trabajo; su adopción como práctica de ingeniería está documentada, no obstante, en catálogos de patrones de la industria, como el del *Microsoft Azure Architecture Center* (Microsoft, 2025). La caracterización detallada de esta variante, su fundamento y su compatibilidad con los invariantes del estilo se desarrollan en las Subsecciones 3.2.5 y 3.2.7.

3.2. El framework como sistema de Tubos y Filtros

Esta sección caracteriza el framework propuesto como un sistema de Tubos y Filtros, aplicando para ello el esquema descriptivo canónico documentado por Cristiá and Pomponio (2025). Las subsecciones que siguen presentan, en este orden, la vista de conjunto del sistema, los filtros que lo componen, los tubos que los conectan, las especializaciones del estilo que se adoptan en el diseño, las deformaciones canónicas aplicables, la verificación de los invariantes esenciales sobre el diseño resultante y un análisis de las ventajas y limitaciones de la elección.

3.2.1. Vista de conjunto y patrón estructural

El diagrama canónico de la Figura 3.1 resume la configuración del sistema: un pipeline lineal de siete filtros que va desde la fuente —el conjunto de portales digitales monitoreados— hasta el sumidero, conformado por la capa de presentación de resultados. Los filtros se denominan Extracción, Enriquecimiento semántico, Composición de contexto, Invocación generativa, Decodificación, Verificación y Normalización; cada uno encapsula una transformación específica sobre la representación de los datos que recibe y la entrega al siguiente filtro a través de un tubo dedicado. Cada tubo transporta un *recordset* tipado —en la terminología de Odo, un conjunto ordenado de referencias a los registros resultantes de la transformación—, mientras que el contenido completo de esos registros reside en un repositorio común compartido por todos los filtros; los puertos de entrada y salida de cada filtro se numeran en la figura y se detallan, junto con su tipo y su cardinalidad, en la tabla de referencia que la acompaña.

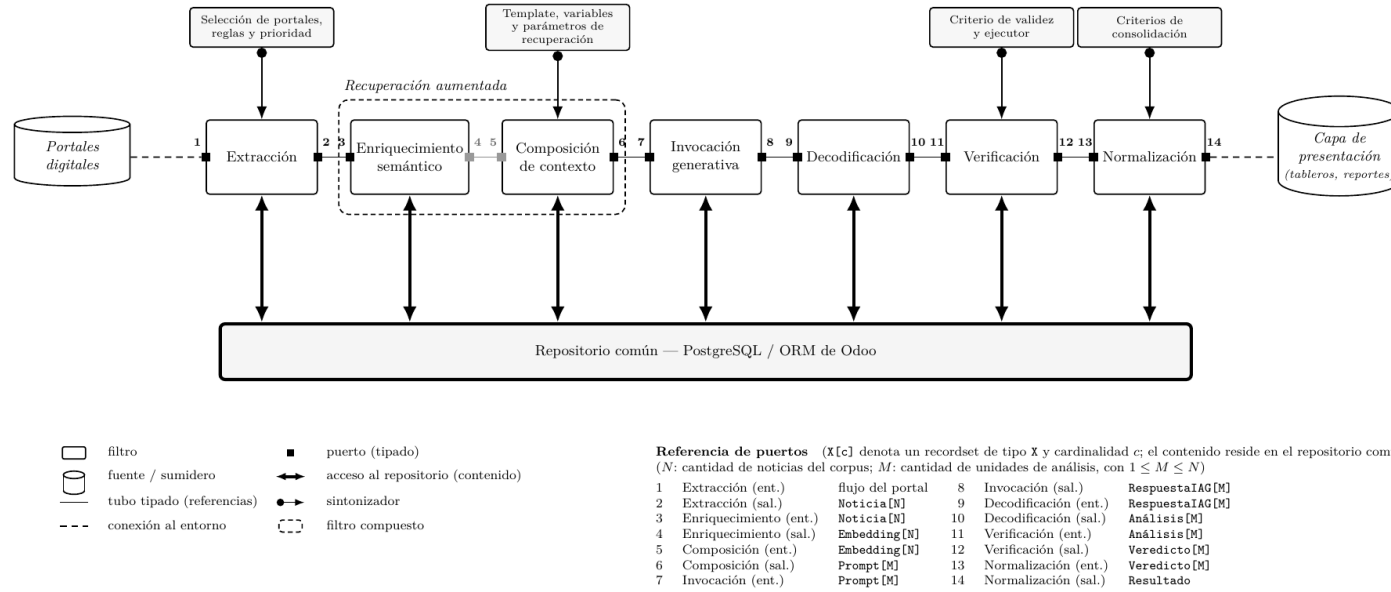


Figura 3.1: Diagrama canónico del framework propuesto como sistema de Tubos y Filtros. La fuente son los portales digitales de noticias y el sumidero es la capa de presentación de resultados (tableros y reportes). Los filtros se comunican mediante tubos tipados que transportan *recordsets* —conjuntos de referencias a registros—, cuyos puertos se numeran y se detallan en la tabla de referencia inferior; el contenido completo de esos registros reside en el repositorio común. Siguiendo la convención del estilo, las conexiones al entorno se representan con línea discontinua y los tubos entre filtros con línea continua —ambos sin punta, dado que el flujo se asume de izquierda a derecha—, en tanto que los sintonizadores —modulares, uno por filtro— se grafican con flecha originada en un círculo y el acceso al repositorio con flecha bidireccional. El recuadro de trazo discontinuo que engloba los filtros de Enriquecimiento semántico y Composición de contexto indica que ambos conforman un filtro compuesto, denominado Recuperación aumentada.

La figura incorpora ya las especializaciones y deformaciones del estilo que el diseño adopta, cuya justificación se desarrolla en las subsecciones siguientes. La configuración se organiza como un *pipeline* —cada filtro posee un único puerto de entrada y un único puerto de salida— de *tubos tipados*, en el que cada tubo declara el tipo del recordset que transporta; ambas especializaciones se fundamentan en la Subsección 3.2.4. Incorpora, además, un repositorio común de datos sobre el cual los filtros leen y escriben el contenido que los tubos no transportan, deformación que se discute en la Subsección 3.2.5. Por último, expone en los filtros que lo requieren *sintonizadores* —procedimientos invocados desde la capa de presentación que incorporan la configuración del experto del dominio sin participar del flujo de datos del pipeline—, tratados en la Subsección 3.2.6.

Tres observaciones sobre la configuración resultan oportunas antes de abordar el detalle de cada componente. En primer lugar, el flujo de datos por la red de tubos es estrictamente acíclico: avanza desde la fuente hacia el sumidero sin ramificaciones ni ciclos. El acceso al repositorio común, de naturaleza bidireccional, no introduce ciclos entre filtros, ya que constituye un mecanismo de almacenamiento compartido y no un tubo que vincule etapas del pipeline entre sí. La iteración del experto del dominio sobre los resultados —fenómeno que tiene lugar entre ejecuciones sucesivas del sistema— es propia del proceso de uso, no de la arquitectura interna del pipeline, y por lo tanto queda fuera del alcance de la presente caracterización. En segundo lugar, fuente y sumidero son componentes externos al framework: los portales digitales no pertenecen al sistema sino que constituyen su entorno de entrada, y la capa de presentación, encargada de la visualización y la generación de reportes, corresponde al entorno de consumo de los resultados. Esta delimitación es coherente con la convención del estilo, según la cual el sistema queda definido por la red de filtros y tubos y no por sus interlocutores externos. En tercer lugar, la configuración exhibe la composicionalidad propia del estilo, presentada en la Subsección 3.1.1: dos de los filtros —Enriquecimiento semántico y Composición de contexto— se agrupan en un filtro compuesto, denominado Recuperación aumentada, que hacia el exterior se comporta como un único filtro. Esta misma propiedad habilita, además, la inclusión, eliminación o sustitución de etapas para acomodar variantes analíticas particulares; la caracterización aquí presentada corresponde al pipeline base que cubre el ciclo completo de análisis sin compromiso con ningún caso de estudio específico.

3.2.2. Componentes: los filtros del framework

Esta subsección caracteriza cada uno de los siete filtros —dos de los cuales, según se anticipó, se agrupan en el filtro compuesto Recuperación aumentada— siguiendo el esquema que el catálogo de Cristiá and Pomponio (2025) prescribe para el componente de un sistema de Tubos y Filtros: para cada filtro se especifican sus puertos de entrada y salida con su tipo, los sintonizadores que expone —si los tiene—, su acceso al repositorio común y una especificación funcional de la transformación que realiza. La numeración de los puertos remite a la tabla de referencia de la Figura 3.1; la estructura interna de los tipos que circulan por ellos se especifica en el Apéndice A.

Todos los filtros comparten un mismo patrón de operación, consecuencia del uso del repositorio común: reciben por su puerto de entrada un *recordset*, recuperan del repositorio el contenido asociado a esas referencias, aplican su transformación, persisten los resultados y emiten por su puerto de salida un nuevo *recordset* que referencia los registros producidos. Salvo el filtro de Extracción —que es un filtro activo, disparado por las acciones planificadas de la plataforma según la frecuencia configurada para cada portal—, los restantes operan a demanda dentro de la ejecución de un análisis, consumiendo los datos disponibles en su puerto de entrada.

Este patrón admite una formulación más precisa, que extiende la notación funcional con que el catálogo describe a los filtros. En el estilo puro, un filtro del pipeline es una función $f : T[c] \rightarrow T'[c']$ de su puerto de entrada en su puerto de salida (Cristiá and Pomponio, 2025, p. 24). En la variante heterogénea aquí adoptada, en cambio, la transformación del contenido ocurre en el repositorio común, de modo que el estado que se transforma es el par flujo–repositorio:

$$f : (T[c], \text{Rep}) \rightarrow (T'[c'], \text{Rep}') \quad \text{con } \text{Rep}' = \text{Rep} \cup \Delta_f,$$

donde Rep es el estado del repositorio al activarse el filtro y Δ_f el conjunto de registros que éste persiste. El *recordset* emitido por el puerto de salida es, precisamente, el conjunto de referencias a ese delta — $\text{refs}(\Delta_f)$ —: el tubo transporta la referencia a la transformación que el repositorio absorbió. Dos consecuencias se siguen de esta formulación. La primera es la *monotonía* del repositorio respecto del flujo de datos: cada filtro únicamente agrega registros — $\text{Rep} \subseteq \text{Rep}'$ —, ninguno modifica ni elimina los que otro produjo, propiedad de la que depende la trazabilidad extremo a extremo del sistema (Apéndice A). La segunda es de método: al admitir el repositorio común, el propio

catálogo exige documentar y explicar el conector adicional que la deformación introduce (Cristiá and Pomponio, 2025, §3.13.1), y la firma extendida constituye esa documentación en los mismos términos funcionales con que el catálogo describe el estilo puro. Conviene precisar el alcance: la firma describe la forma de la transformación, no su semántica —no supone determinismo, pues la invocación generativa es estocástica, ni pureza funcional—. Las siete firmas concretas del pipeline se consignan en el Apéndice A.

Extracción. Interfaz:

- **Puerto de entrada** (1): flujo del portal, como conexión al entorno desde la fuente.
- **Puerto de salida** (2): `Noticia[]`.
- **Sintonizador**: selección de los portales a monitorear, definición de las reglas de extracción mediante expresiones regulares y asignación de la prioridad de monitoreo.
- **Acceso al repositorio**: escribe los registros `Noticia` (título, copete, texto y enlace de la publicación, junto con sus metadatos de procedencia; Apéndice A).

Especificación funcional. Accede a los portales registrados, recorre su estructura mediante técnicas de *web scraping* de tipo *spider* y recupera las publicaciones que satisfacen las reglas de extracción configuradas. De cada publicación extrae los elementos relevantes, normaliza el texto y lo persiste como un registro `Noticia` en el repositorio. Emite por su puerto de salida el *recordset* con la totalidad de las noticias que conforman el corpus del análisis. La descripción detallada de esta etapa, que se apoya en la herramienta de scraping preexistente del entorno institucional, se desarrolla en la Subsección 3.3.1.

Recuperación aumentada (filtro compuesto). Los filtros de Enriquecimiento semántico y Composición de contexto no operan de manera independiente: el primero genera las representaciones vectoriales que el segundo consume para recuperar el material empírico relevante. En conjunto implementan el patrón de *recuperación aumentada* (*retrieval-augmented*): la indexación semántica del corpus y su posterior recuperación contextual. Por esta

razón se los agrupa en un único filtro compuesto que, por el cierre composicional del estilo (Subsección 3.1.1), constituye en sí mismo un filtro: hacia el exterior expone un puerto de entrada de tipo `Noticia[]` (puerto 3) y un puerto de salida de tipo `Prompt[]` (puerto 6), mientras que el tubo que comunica ambos sub-filtros (puertos 4 y 5), de tipo `Embedding[]`, queda confinado a su interior. La denominación recoge el concepto homónimo de la inteligencia artificial generativa contemporánea, con la salvedad de que la etapa de generación propiamente dicha corresponde al filtro de Invocación generativa, posterior a este compuesto. Sus dos sub-filtros se describen a continuación.

Enriquecimiento semántico (sub-filtro). Interfaz:

- **Puerto de entrada (3):** `Noticia[]`.
- **Puerto de salida (4):** `Embedding[]`.
- **Acceso al repositorio:** lee el texto de las noticias recibidas y escribe los registros `Embedding` —la representación vectorial asociada a cada una—.

Especificación funcional. Para cada noticia del *recordset* de entrada, recupera su contenido textual del repositorio y computa su representación vectorial mediante el modelo de *embedding* configurado, persistiéndola como un registro `Embedding` que referencia a la noticia que codifica. Preserva la cardinalidad —emite un `Embedding` por noticia— pero transforma el tipo que circula: su salida no es el conjunto de noticias sino el de sus representaciones semánticas, disponibles para las operaciones de recuperación que realiza el filtro siguiente. Este filtro no expone sintonizadores al experto del dominio: su configuración —el modelo de *embedding* y la estrategia de fragmentación— es de carácter técnico y la fija el implementador como parámetro interno. Conviene distinguir la *generación* del *embedding*, que ocurre aquí sin intervención del experto, de su posterior *uso* en la recuperación, que sí admite parametrización de dominio y se realiza en el filtro de Composición de contexto.

Composición de contexto (sub-filtro). Interfaz:

- **Puerto de entrada (5):** `Embedding[]`.
- **Puerto de salida (6):** `Prompt[]`.

- **Sintonizador:** selección del *template* de *prompt*, customización de las variables de contexto, ingreso de texto libre, ajuste de los parámetros del modelo generativo (temperatura, longitud de salida) y de los parámetros de recuperación (cantidad de documentos a recuperar y umbral de similitud).
- **Acceso al repositorio:** lee los vectores recibidos y las noticias que estos referencian —para la recuperación— y los *templates* disponibles; escribe los registros **Prompt**.

Especificación funcional. Constituye el núcleo de la intervención del experto del dominio. A partir del *template* seleccionado y de las variables de contexto configuradas, realiza la recuperación contextual (*retrieval*): a partir de los **Embedding** recibidos consulta el repositorio para obtener las noticias semánticamente relevantes según similitud vectorial, e integra ese material empírico con el *template*, las aclaraciones de texto libre y los parámetros definidos para componer el *prompt* completo destinado al modelo generativo. La cardinalidad puede transformarse en esta etapa: a partir del *recordset* recibido, el filtro produce uno o varios *prompts* según la estrategia analítica —un único *prompt* agregado sobre el corpus, o un *prompt* por unidad de análisis—. Persiste los registros **Prompt** y emite el *recordset* correspondiente. El diseño del panel de *templates* que da soporte a este sintonizador se desarrolla en la Subsección 3.2.6.

Invocación generativa. Interfaz:

- **Puerto de entrada (7):** **Prompt** [] .
- **Puerto de salida (8):** **RespuestaIAG** [] .
- **Acceso al repositorio:** lee los *prompts* y escribe las respuestas crudas del modelo.

Especificación funcional. Para cada *prompt* del *recordset* de entrada, establece la conexión con la interfaz de programación (API) del modelo generativo configurado, ejecuta la petición y recibe la respuesta producida. Persiste cada respuesta cruda como un registro **RespuestaIAG** y emite el *recordset* resultante. La selección del proveedor, del modelo y de las credenciales de acceso a la API es configuración técnica del implementador, por lo que el filtro no expone sintonizadores al experto. El aislamiento de esta etapa en un filtro

propio —cuyo único cometido es la interacción con el servicio generativo externo— es lo que materializa la propiedad de sustituibilidad analizada en la Subsección 3.1.3: cambiar de proveedor o de modelo se reduce a reemplazar este filtro, sin impacto sobre el resto del pipeline.

Decodificación. Interfaz:

- **Puerto de entrada (9):** `RespuestaIAG[]`.
- **Puerto de salida (10):** `Análisis[]`.
- **Acceso al repositorio:** lee las respuestas crudas y escribe los análisis estructurados.

Especificación funcional. Transforma la salida del modelo generativo —típicamente texto en un lenguaje de marcado— en una estructura de datos del dominio apta para su tratamiento posterior. Para cada respuesta cruda del *recordset* de entrada, interpreta su formato —aísla el fragmento estructurado de la prosa circundante y lo convierte de texto en objeto—, valida su conformidad con la estructura esperada —el esquema declarado por el *template* del estudio, que el filtro alcanza siguiendo la cadena de referencias que va de la respuesta a su *prompt* y de éste a su *template*, sin requerir por ello configuración de dominio propia—, canonicaliza los valores hacia los tipos del dominio y persiste el resultado como un registro **Análisis**. La validación opera como una compuerta: la respuesta que no conforma el esquema no se persiste como **Análisis** —la política ante la no conformidad (reintentar la invocación, reparar, descartar) es configuración técnica del implementador—, de modo que las etapas posteriores consumen exclusivamente estructuras válidas. Emite el *recordset* de análisis estructurados. Esta separación entre la obtención de la respuesta (filtro anterior) y su decodificación aísla el manejo del formato de salida del modelo, de modo que un cambio en dicho formato no afecte a las etapas previas.

Conviene precisar el sentido del término y el compromiso arquitectónico que la etapa encierra. En el procesamiento de lenguaje natural, *decoding* designa la estrategia con que el modelo selecciona cada token durante la generación —búsqueda voraz, *beam search*, muestreo— (Wiher et al., 2022); la decodificación de este filtro es, en cambio, la interpretación estructural de la salida ya generada. La práctica contemporánea conoce un punto en el que ambos sentidos se fusionan: la *generación restringida* (*constrained decoding*)

—hoy la técnica dominante en la industria para obtener salida estructurada— hace que una gramática o un autómeta enmascare el vocabulario del modelo en cada paso de la generación, garantizando por construcción la validez sintáctica del resultado (Willard and Louf, 2023; Geng et al., 2025). El diseño aquí adoptado no excluye esa técnica, pero no deposita en ella el contrato: la garantía que ofrece vive en el momento de la invocación y se acopla al proveedor o al motor de inferencia —los proveedores comerciales soportan deliberadamente sólo un subconjunto de los esquemas expresables (Geng et al., 2025)—, mientras que mantener la validación aguas abajo, en un filtro propio, la conserva agnóstica del proveedor y preserva la sustituibilidad de Invocación generativa (Subsección 3.1.3). Ambas estrategias son, además, componibles: aun si el implementador emplea generación restringida en la invocación, la validación estructural contra el esquema del estudio, la canonicización y la compuerta de no conformidad permanecen en esta etapa, pues exceden lo que una gramática puede garantizar. Conviene delimitar el alcance de esta etapa: Decodificación garantiza que el análisis sea *estructuralmente* conforme —que tenga la forma que el esquema declara—, pero no que su *contenido* sea fiel al material empírico del que dice derivar. Esa segunda garantía —de otra naturaleza— es la que asume el filtro siguiente.

Verificación. Interfaz:

- **Puerto de entrada (11):** `Análisis[]`.
- **Puerto de salida (12):** `Veredicto[]`.
- **Sintonizador:** definición del criterio de validez —qué se considera un análisis fiel a la evidencia— y selección del ejecutor de la verificación (Subsección 3.2.6).
- **Acceso al repositorio:** lee cada análisis estructurado y, siguiendo la cadena de referencias, el material empírico que lo sustenta —las noticias que su *prompt* incorporó por recuperación semántica—; escribe los registros **Veredicto**.

Especificación funcional. Contrasta cada **Análisis** con la evidencia empírica de la que dice derivar y emite un juicio sobre su fidelidad. Para cada análisis del *recordset* de entrada, recupera del repositorio las noticias que su *prompt* incorporó por recuperación semántica y evalúa en qué medida las afirmaciones del análisis están efectivamente sostenidas por ese material —señalando,

en particular, las afirmaciones no ancladas en la evidencia, esto es, las alucinaciones del modelo generativo—. Persiste el resultado como un registro **Veredicto** que referencia al análisis juzgado, califica su estado —válido, observado o no sustentado—, cuantifica la confianza asociada y enumera los fragmentos de evidencia que respaldan o contradicen cada afirmación. Preserva la cardinalidad —emite un **Veredicto** por análisis— pero transforma el tipo que circula: aguas abajo ya no viaja el análisis sino el juicio sobre su fidelidad, del que aquél pende por referencia.

La verificación opera como una *compuerta blanda*: a diferencia de la compuerta de Decodificación —que descarta lo que no conforma el esquema—, aquí ningún análisis se suprime; los que no superan el criterio de validez se marcan y se propagan señalados, de modo que la decisión editorial última quede en manos del analista y no de un automatismo. Esta política responde de manera directa a tres brechas identificadas en la revisión sistemática (Sección 2.4): la fidelidad de la evidencia extraída —la dificultad de auditar qué fragmentos apoyan cada afirmación generada—, la calibración de la incertidumbre en los *scores* de análisis —que aquí se reifica en la confianza que el veredicto cuantifica— (Pavlyshenko, 2024), y la gobernanza editorial mediante procedimientos *human-in-the-loop* ante contenidos controvertidos. Que la compuerta sea blanda, y no una criba automática, es lo que la hace coherente con esta última brecha: el filtro no publica ni censura, sino que provee al experto el juicio fundamentado sobre el cual decidir.

Dos precisiones sobre su lugar en la arquitectura. En primer lugar, la verificación no reintroduce ciclos en la red de tubos: cuando un veredicto adverso motiva regenerar un análisis, esa reejecución es un fenómeno del proceso de uso —que tiene lugar entre corridas sucesivas del sistema, según se precisó en la Subsección 3.2.1— y no un tubo de retorno que vincule este filtro con los anteriores; el flujo interno permanece estrictamente acíclico. En segundo lugar, el filtro es agnóstico respecto de quién ejecuta la verificación: por la sustituibilidad que las invariantes 1 y 2 garantizan (Subsección 3.1.3), su realización interna —un procedimiento automático que asiste a un revisor humano, o un conjunto de agentes generativos que confrontan sus juicios— puede reemplazarse sin alterar su interfaz de puertos ni el resto del pipeline. El criterio de validez y la elección del ejecutor se incorporan por sintonizador, no por el flujo de datos, de modo que el aporte del experto —o la orquestación de los agentes— se mantiene fuera de la red de tubos, en línea con la frontera que la Subsección 3.2.6 establece para toda configuración de dominio.

Normalización. Interfaz:

- **Puerto de entrada (13):** Veredicto[].
- **Puerto de salida (14):** Resultado.
- **Sintonizador:** revisión del analista, definición de los criterios de consolidación —incluida la política de tratamiento de los análisis observados por la verificación— y de las variables de comparación contra las cuales se evalúan los resultados.
- **Acceso al repositorio:** lee los veredictos y, por referencia, los análisis que estos juzgan, y escribe el resultado consolidado del estudio.

Especificación funcional. Consolida el *recordset* de veredictos —y los análisis verificados que estos referencian— en un único registro **Resultado** que representa el producto del estudio. Documenta el recorrido del proceso, organiza los datos de manera coherente, aplica la política de consolidación definida por el analista —que determina cómo se integran los análisis observados por la verificación—, compara los análisis con las variables definidas y produce la agregación que da sentido al análisis del *agenda-setting* a escala de corpus. Es en este filtro donde la cardinalidad se reduce de un conjunto de veredictos a un resultado único consolidado, que se entrega a la capa de presentación —el sumidero del sistema— para su visualización en tableros y reportes.

3.2.3. Conectores: tubos tipados

Los tubos son los conectores del sistema: canales unidireccionales que enlazan el puerto de salida de un filtro con el puerto de entrada del siguiente, preservan el orden de los elementos que transportan y no modifican su contenido (Subsección 3.2.7). Cada tubo está tipado —transporta un *recordset* de un tipo declarado (Subsección 3.2.4)—: un conjunto ordenado de referencias cuyo contenido reside en el repositorio común (Subsección 3.2.5).

La caracterización de un tubo no se agota, sin embargo, en el tipo de sus elementos: comprende también su *cardinalidad*, esto es, el tamaño del *recordset* que transporta. La cardinalidad no se conserva a lo largo de la cadena: la mayoría de las etapas la preservan, la composición de contexto la redefine —al producir uno o varios *prompts* a partir del conjunto de noticias— y la normalización la reduce a la unidad —al consolidar los análisis en un

único resultado—. Por ello, la firma de cada tubo se expresa como **Tipo**[*c*], donde *c* denota la cardinalidad del *recordset* transportado.

El pipeline comprende seis tubos internos, además de las conexiones de la fuente y el sumidero con el entorno. La cardinalidad recorre la cadena $N \rightarrow N \rightarrow M \rightarrow M \rightarrow M \rightarrow 1$, cuyos tramos se detallan a continuación con remisión a los puertos de la Figura 3.1.

Extracción produce el corpus: emite **Noticia**[*N*] (puerto 2), donde *N* es la cantidad de noticias recolectadas; la conexión previa —del portal al puerto 1— transporta el flujo crudo del portal, sin estructura de *recordset*, y queda por ello fuera de la notación tipada. El primer tubo (2→3) lleva ese **Noticia**[*N*] a Enriquecimiento semántico, que preserva la cardinalidad pero transforma el tipo: emite un **Embedding** por cada noticia (puerto 4), de modo que el tubo interno del filtro compuesto *Recuperación aumentada* (4→5) transporta **Embedding**[*N*] —las *N* representaciones vectoriales, cada una con la referencia a la noticia que codifica—. En este tramo la cardinalidad es la invariante y la representación es lo que cambia: cada noticia sigue presente en el flujo, pero mediada por su codificación semántica.

Composición de contexto redefine la cardinalidad: a partir de los *N* **Embedding** recibidos emite **Prompt**[*M*] (puerto 6), con $1 \leq M \leq N$, donde *M* es la cantidad de *unidades de análisis* que fija la estrategia —desde un único *prompt* agregado sobre el corpus ($M = 1$) hasta un *prompt* por noticia ($M = N$)—. Cada **Prompt** incorpora, por recuperación semántica, el subconjunto de noticias relevante a su unidad; la relación entre noticias y *prompts* es, por lo tanto, de muchos a muchos, y se registra en el propio **Prompt** (Subsección 3.2.2).

Los tres tubos siguientes preservan *M*: Invocación generativa emite una **RespuestaIAG** por cada *prompt* (**Prompt**[*M*] → **RespuestaIAG**[*M*], puerto 8), Decodificación una **Análisis** por cada respuesta (**RespuestaIAG**[*M*] → **Análisis**[*M*], puerto 10) y Verificación un **Veredicto** por cada análisis (**Análisis**[*M*] → **Veredicto**[*M*], puerto 12). Que la cardinalidad se mantenga en *M* a lo largo de estas etapas expresa que el análisis procede como un *map* sobre las unidades definidas en la composición: cada unidad se invoca, se decodifica y se verifica de manera independiente.

Finalmente, Normalización reduce la cardinalidad a la unidad: consolida los *M* **Veredicto** —y los análisis verificados que referencian— en un único registro **Resultado** (puerto 14) que, por ser un registro único y no un *recordset*, se tipa sin cardinalidad. Esta reducción es el *reduce* que clausura el *map* de las etapas previas. Vista en conjunto, la dinámica de cardinalidad

del pipeline —seleccionar M unidades sobre N noticias, procesarlas de forma independiente y consolidarlas en un único resultado— es lo que permite escalar el análisis a corpus que no cabrían en una sola invocación del modelo generativo.

3.2.4. Especializaciones aplicadas: pipelines y tubos tipados

El diseño adopta dos de las especializaciones canónicas que el estilo admite —documentadas tanto en la formulación seminal (Garlan and Shaw, 1994) como en el catálogo de Cristiá and Pomponio (2025, §3.12)—: la de *pipelines* y la de *tubos tipados*, aplicadas de manera combinada.

La especialización de *pipeline* restringe la topología a una secuencia lineal de filtros, cada uno con un único puerto de entrada y un único puerto de salida (Cristiá and Pomponio, 2025, §3.12.1); en los términos de Garlan and Shaw (1994), “pipelines...restrict the topologies to linear sequences of filters”. Su adopción responde a la naturaleza del problema: el análisis de noticias procede como una cadena de transformaciones sucesivas —extracción, enriquecimiento, composición, invocación, decodificación y normalización— sin ramificaciones ni puntos de decisión en su configuración base, de modo que la restricción a un único puerto de entrada y de salida por filtro no supone pérdida de expresividad para el ciclo completo de análisis. Las variantes analíticas que requieran recombinar etapas se acomodan por la composicionalidad del estilo (Subsección 3.1.1), no por la ramificación de la topología.

La especialización de *tubos tipados* exige que cada tubo transporte un dato de tipo bien definido —“typed pipes, which require that the data passed between two filters have a well-defined type” (Garlan and Shaw, 1994)—, de manera que un puerto de tipo T sólo pueda conectarse a un tubo del mismo tipo, sin que el conector realice conversión alguna (Cristiá and Pomponio, 2025, §3.12.2). En el framework, cada tubo declara el tipo del *record-set* que transporta —`Noticia[]`, `Embedding[]`, `Prompt[]`, `RespuestaIAG[]`, `Análisis[]`, `Veredicto[]` y `Resultado`, según la tabla de referencia de la Figura 3.1; su estructura interna se especifica en el Apéndice A—. El tipado cumple una doble función: documenta el contrato entre etapas —qué produce cada filtro y qué espera el siguiente— y permite detectar tempranamente errores de composición, al volver la conexión de dos filtros incompatibles un error de tipo antes que un fallo en tiempo de ejecución. Conviene señalar el

compromiso que supone: al declarar tipos concretos, los filtros dependen más del contexto que en la formulación no tipada, lo que reduce la flexibilidad de recomposición a cambio de mayor seguridad y de un tratamiento más ordenado de los datos (Cristiá and Pomponio, 2025, §3.12.2).

3.2.5. Deformación aplicada: repositorio común sobre PostgreSQL

La caracterización del framework incorpora una deformación canónica del estilo: un *repositorio común* de datos, materializado sobre PostgreSQL a través del ORM de Odoo, del cual todos los filtros leen y sobre el cual escriben el contenido que los tubos no transportan (Cristiá and Pomponio, 2025, §3.13.1). Por los tubos no circula el contenido completo de las noticias, los *prompts* o los análisis, sino *recordsets* —conjuntos ordenados de referencias a los registros—, mientras que el contenido reside en el repositorio y se recupera bajo demanda.

Esta separación entre referencia y contenido no es una imposición sobre la plataforma, sino la semántica nativa del ORM de Odoo. La documentación oficial define que “every model instance is a ‘recordset’, i.e., an ordered collection of records of the model” y que el método `browse` “returns a recordset for the ids provided as parameter” —de modo que un *recordset* se representa por sus identificadores: `browse([7, 18, 12])` produce `res.partner(7, 18, 12)`—. El contenido de los campos no viaja con la referencia: la propia plataforma “maintains a cache for the fields of the records, so that not every field access issues a database request”, recuperándolo del repositorio sólo al accederlo (Odoo S.A., 2024). Transmitir *recordsets* por los tubos equivale, entonces, a transmitir referencias tipadas cuyo contenido se materializa contra el repositorio común.

A nivel del conector, este esquema instancia el patrón *Claim Check* (Hohpe and Woolf, 2003): “store message data in a persistent store and pass a Claim Check to subsequent components”. Lejos de ser una mera recomendación de catálogo, el patrón ha sido reificado en la literatura académica sobre patrones de integración empresarial: al reexpresar el catálogo de Hohpe and Woolf (2003) en una notación de modelado formal, Ritter (2014) caracteriza el Claim Check como un *call by reference* sobre la carga útil del mensaje —el contenido se almacena en un repositorio de datos y sólo la referencia se transmite al siguiente elemento del flujo—. El mismo principio —transportar una

referencia y no la carga útil— es práctica corriente en los orquestadores de *pipelines* de datos y de aprendizaje automático: Apache Airflow recomienda comunicar por su canal sólo mensajes pequeños y referenciar a un almacenamiento remoto los datos mayores, empujando la ruta del dato y no el dato (Apache Software Foundation, 2026), y Kubeflow Pipelines distingue los parámetros, pasados por valor, de los *artifacts*, que son una referencia (`.uri`) al objeto en un almacén externo (The Kubeflow Authors, 2026).

Conviene precisar el alcance de esta decisión. En los sistemas mencionados, el reparto entre referencia y valor suele gobernarse por un umbral de tamaño —los datos pequeños viajan en línea por el conector y sólo los grandes se referencian al almacén—, configurando arquitecturas híbridas. El diseño aquí presentado adopta, en su configuración base, el paso uniforme por referencia, decisión que se apoya en que el repositorio común constituye la única fuente de verdad del sistema y en que la propia plataforma representa sus colecciones como recordsets referenciales. La adopción de un repositorio junto a la red de tubos no abandona el estilo de Tubos y Filtros, sino que configura una *arquitectura heterogénea* en el sentido de Garlan and Shaw (1994), en la que los componentes acceden a un repositorio por parte de su interfaz pero interactúan a través de tubos.

3.2.6. Sintonizadores

El catálogo de Cristiá and Pomponio (2025, §3.13.2) documenta los *sintonizadores* como una deformación canónica del estilo: procedimientos que un filtro expone para que un componente externo —típicamente una interfaz gráfica— altere o ajuste su comportamiento sin participar del flujo de datos del sistema. El diseño reifica esta deformación para modelar la intervención del experto del dominio, que parametriza el comportamiento de ciertos filtros sin inyectar datos en el pipeline.

Esta caracterización no es exclusiva del catálogo: tiene un precedente directo en la formulación seminal del estilo. Al resolver la interacción con el usuario en su caso de estudio del osciloscopio, Garlan and Shaw (1994) asocian a cada filtro una *control interface* “that allows an external entity to set parameters of operation for the filter”, y modelan así los filtros como funciones de orden superior “for which the configuration parameters determine what data transformation the filter will perform”; su virtud es que “decouples the signal processing functions...from the actual user interface”. El sintonizador reifica esa interfaz de control: desacopla la parametrización que aporta

el experto del procesamiento que realiza el filtro. La misma separación entre configuración y datos reaparece en las bibliotecas modernas de aprendizaje automático —en *scikit-learn*, el constructor de un estimador “does not see any actual data...All it does is attach the given parameters to the object” (Buitinck et al., 2013)—.

En el framework exponen sintonizador únicamente los cuatro filtros cuyo comportamiento depende de decisiones de dominio: Extracción (selección de portales, reglas de extracción y prioridad de monitoreo), Composición de contexto (selección del *template*, variables de contexto, texto libre y parámetros de recuperación y del modelo generativo), Verificación (criterio de validez de los análisis y selección del ejecutor de la verificación) y Normalización (criterios de consolidación y variables de comparación). Los filtros restantes —Enriquecimiento, Invocación y Decodificación— no lo hacen: su configuración es de carácter técnico y la fija el implementador como parámetro interno (Subsección 3.2.2). Todos los sintonizadores se invocan desde la capa de presentación —nunca desde otro filtro— y se grafican en la Figura 3.1 con la flecha originada en un círculo que apunta a cada filtro configurable. El más elaborado es el de Composición de contexto, que se materializa como un panel de creación de *templates* de *prompt*: un subsistema gestionado que habilita al experto configurar, parametrizar y reutilizar plantillas en lugar de resolver esa lógica de manera *ad hoc*, como es habitual en las aplicaciones basadas en modelos de lenguaje. Cada *template* declara el objetivo del análisis —alcance, paradigma de análisis, referencias a considerar—, las instrucciones parametrizadas con variables de contexto y el esquema de la respuesta esperada, reificando los datos de entrada para la construcción de *prompts* anticipados en el Capítulo 1; su estructura se especifica en el Apéndice A, y su realización concreta en el módulo se retoma en la Subsección 3.3.3.

Que la intervención del experto se canalice por sintonizadores —y no por el flujo de datos— es lo que la mantiene compatible con el estilo: el sintonizador ajusta *cómo* transforma un filtro, sin alterar la red de tubos ni agregar un puerto al pipeline. La configuración así incorporada es un parámetro del filtro, no un dato que circule por la arquitectura.

3.2.7. Verificación de los invariantes esenciales

Esta subsección verifica que el diseño resultante —un *pipeline* de tubos tipados con repositorio común— satisface los cuatro invariantes esenciales enunciados en la Subsección 3.1.2.

Identidad de los filtros (invariante 1). Ningún filtro referencia a otro por su identidad. La comunicación se produce exclusivamente a través de los puertos tipados —por los que circulan recordsets— y del repositorio común, sobre el que cada filtro lee y escribe registros sin conocer qué filtro los produjo ni cuál los consumirá. La sustituibilidad analizada en la Subsección 3.1.3 es consecuencia directa de esta propiedad.

Estado interno de los filtros (invariante 2). Ningún filtro accede al estado interno de otro. El repositorio común es un almacén externo compartido —conforme al patrón Claim Check— y no el estado en ejecución de ningún filtro; que se materialice sobre la base de datos de Odoo es una particularidad de la implementación que no incide en esta propiedad.

Orden de cómputo (invariante 3). La corrección del resultado no depende del orden en que cada filtro realiza sus cálculos internos. Las dependencias que la topología establece —un filtro no puede materializar un contenido que el filtro anterior aún no persistió— son dependencias de flujo propias del grafo de tubos, no dependencias del *scheduling* interno, según se precisó en la Subsección 3.1.2. Disponibles sus entradas, cada filtro produce el mismo resultado con independencia del orden relativo de cómputo.

Integridad del transporte (invariante 4). Los tubos transportan recordsets —referencias— sin alterarlos; toda transformación del contenido ocurre dentro de los filtros y queda persistida en el repositorio. El conector no realiza conversión ni modificación alguna sobre lo que transmite, en línea con la especialización de tubos tipados adoptada en la Subsección 3.2.4.

3.2.8. Ventajas y limitaciones de la elección

La elección del estilo se apoya en tres ventajas ya fundamentadas en la Subsección 3.1.3, que aquí sólo se recapitulan: la *sustituibilidad* de los filtros —que permite actualizar la extracción o reemplazar el modelo generativo sin reescribir el resto del pipeline—, la *auditabilidad lineal* del flujo —que localiza cada transformación en un filtro identificable, condición para la trazabilidad de la evidencia— y el *cierre composicional* —que habilita recombinar etapas para distintos casos de estudio—. Sobre esas ventajas se construye el aporte del capítulo; resta señalar las limitaciones y compromisos que la elección concreta acarrea.

En primer lugar, el diseño no es una instancia pura del estilo, sino una arquitectura heterogénea que incorpora un repositorio común (Subsección 3.2.5). El repositorio es un componente del que todos los filtros dependen: acopla

el pipeline a la plataforma de persistencia —Odoo sobre PostgreSQL— y concentra en un único almacén la disponibilidad del contenido. Este acoplamiento es el precio de las ventajas que el repositorio aporta —evitar la copia de grandes volúmenes por los tubos y disponer de una única fuente de verdad— y debe asumirse como una decisión de diseño antes que como una propiedad del estilo canónico.

En segundo lugar, la decisión de transportar *siempre* referencias por los tubos —y no un reparto híbrido por tamaño, como es habitual en los orquestadores relevados (Subsección 3.2.5)— privilegia la uniformidad y la coherencia con el modelo referencial de Odoo, pero introduce un acceso al repositorio en cada etapa aun cuando el volumen del dato no lo exigiría. Es un compromiso favorable en el contexto de la plataforma elegida, no una optimización universal.

En tercer lugar, el framework se aparta del caso de uso paradigmático del estilo —el procesamiento incremental de un *stream*—: opera sobre *recordsets* que representan corpus completos, y varias de sus etapas no son incrementales (la composición agrega la cardinalidad y la normalización la reduce a un único resultado). Garlan and Shaw (1994) señalan que, en su organización *batch*, el estilo resulta menos apropiado para interacciones incrementales; la presente elección es coherente con un procesamiento por lotes —acorde al análisis de un corpus de noticias— y no con la vigilancia en tiempo real, escenario que excedería la configuración base aquí caracterizada.

3.3. Implementación: la solapa en Odoo

3.3.1. Infraestructura preexistente del entorno CAE-TI/CIM

3.3.2. Stack tecnológico

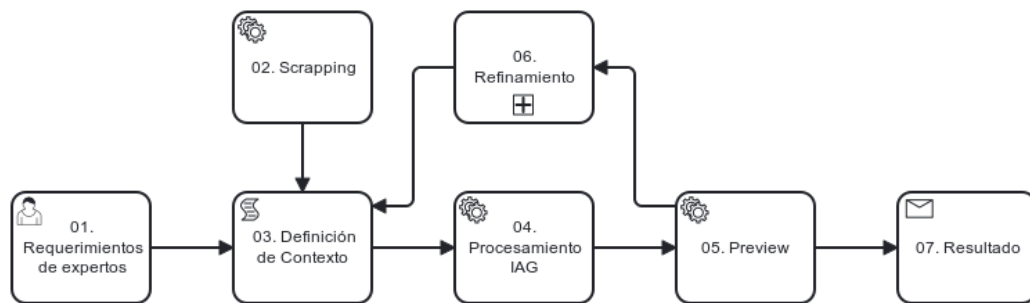
3.3.3. Materialización: mapeo de filtros y tubos al módulo Odoo

3.4. Síntesis del capítulo

Capítulo 4

Mapa de procesos

4.1. Proceso general



Objetivo: Crear un framework para automatizar el análisis de noticias digitales a través de inteligencia artificial generativa.

Alcance: Desarrollar un framework que permita recopilar requerimientos de expertos (01), recopilar datos del mundo real utilizando técnicas de *scrapping* (02), definir el contexto del análisis mediante el uso de *prompt engineering* (03), obtener la salida de la IAG (04), realizar una visualización previa de los resultados (05), refinar la salida a través de la retroalimentación del análisis, ajustando y mejorando el contexto de manera iterativa; adaptándose progresivamente a las necesidades del experto a cargo (06), y presentar los resultados obtenidos (07).

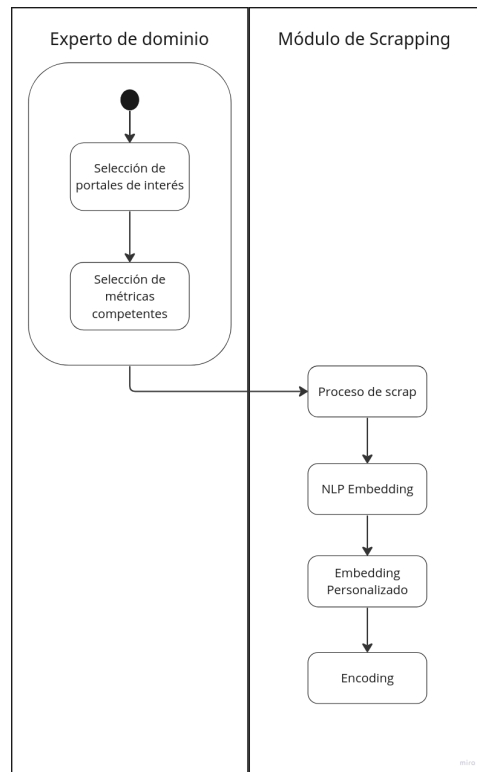
Entrada: Datos empíricos de las noticias junto con información de contexto, requerimientos de análisis y tipo de salida deseada.

Salida: Resultados producidos por la IAG, que pueden ser en forma de narrativas, tablas, gráficos u otras representaciones visuales, dependiendo de los objetivos requeridos.

De los procesos antes mencionados, a continuación se describen los procesos de soporte Nro. 02 -**Scrapping**, 03 - **Definición de Contexto** y 04- **Procesamiento IAG**.

4.2. Recopilación de requerimientos

4.3. Scrapping



Objetivo: Recolectar datos relevantes de resultados electorales, encuestas y percepción de la gente en portales digitales regionales. Además, se busca automatizar este proceso para hacerlo más eficiente y facilitar el análisis posterior de los datos extraídos.

Alcance: Selección de los portales digitales regionales más pertinentes para la obtención de datos. Por otro lado, se diseñarán formularios que permitan al experto especificar variables clave, como el rango de fechas y el tipo de elección (provincial o nacional).

Entrada: Portales digitales regionales relevantes y variables claves como el rango de fechas y el tipo de elección.

Salida: Datos procesados y transformados utilizando técnicas de NPL embedding, que permitirán su representación en forma numérica. Además, se aplicarán técnicas de encoding para almacenar los datos en un formato adecuado, teniendo en cuenta su posterior análisis y la implementación de algoritmos de inteligencia artificial.

Inicio

01. Selección de portales de interés

Actor: Experto de dominio.

Actividad: Seleccionar los portales que serán scrapeados por la herramienta. Esta elección se basa en criterios de idoneidad, credibilidad, número de visitas de usuarios y región geográfica específica. El experto evaluará cuidadosamente cada portal para determinar su relevancia y confiabilidad en cuanto a los datos que se desean obtener. Esta selección se basa en el conocimiento y experiencia del experto en el dominio específico y busca garantizar que los portales elegidos aporten información valiosa y representativa para el análisis posterior del proceso de Scrapping.

02. Selección de métricas competentes

Actor: Experto de dominio.

Actividad: Acotar la información que será recopilada por la aplicación de Scrapping. Esto implica seleccionar las métricas y datos relevantes que se guardarán durante el proceso de extracción. Además de las palabras clave y los títulos importantes, el experto puede considerar la inclusión de otros elementos, como:

Categorías de información específicas relacionadas con los resultados electorales, encuestas o percepciones de la gente. Rango de fechas a recorrer para obtener datos históricos o analizar tendencias a lo largo del tiempo. Variables

demográficas o geográficas relevantes para segmentar y comprender mejor los datos recolectados. Otros criterios específicos basados en el objetivo y alcance de la investigación. El experto debe tener en cuenta que la selección de métricas competentes es fundamental para garantizar la calidad y relevancia de los datos extraídos durante el proceso de Scrapping.

03. Proceso de scrapp

Actor: Aplicación de Scrapping.

Actividad: Recorrer los portales seleccionados y guardar la información relevante. Utiliza técnicas automatizadas para navegar por las páginas web, identificar los datos definidos previamente y extraerlos. La información se almacena en un formato adecuado para su posterior procesamiento y análisis.

04. NPL Embedding

Actor: Aplicación de Scrapping.

Actividad: La aplicación utilizará técnicas de NPL (Natural Language Processing) Embedding para transformar los datos de texto recolectados en representaciones numéricas de alta dimensionalidad. Esto implica asignar un vector numérico a cada palabra o frase, capturando su significado y contexto semántico. El objetivo es organizar y estructurar de manera que las palabras similares se encuentren en un espacio vectorial cercano, permitiendo un análisis más eficiente y preciso en etapas posteriores del proceso. El NPL Embedding ayuda a la aplicación a comprender y procesar el texto de manera más sofisticada, facilitando la identificación de patrones, temas o tendencias en los datos extraídos.

05. Embedding Personalizado

Actor: Experto de dominio.

Actividad: Acotar o mejorar el embedding clásico aplicado a la data estructurada, de manera que pueda ser procesada de manera más efectiva por una inteligencia artificial generativa. El experto puede ajustar parámetros, aplicar técnicas avanzadas de embedding o incluso desarrollar nuevos enfoques para adaptar el embedding al contexto específico de la investigación. Puede involucrar la definición de reglas adicionales, ajustes finos en los pesos asignados a ciertos elementos o la inclusión de información adicional que sea relevante para el análisis. Este subproceso puede ser realizado de forma manual por el experto o, en algunos casos, ser apoyado por herramientas de

automatización desarrolladas específicamente para este propósito.

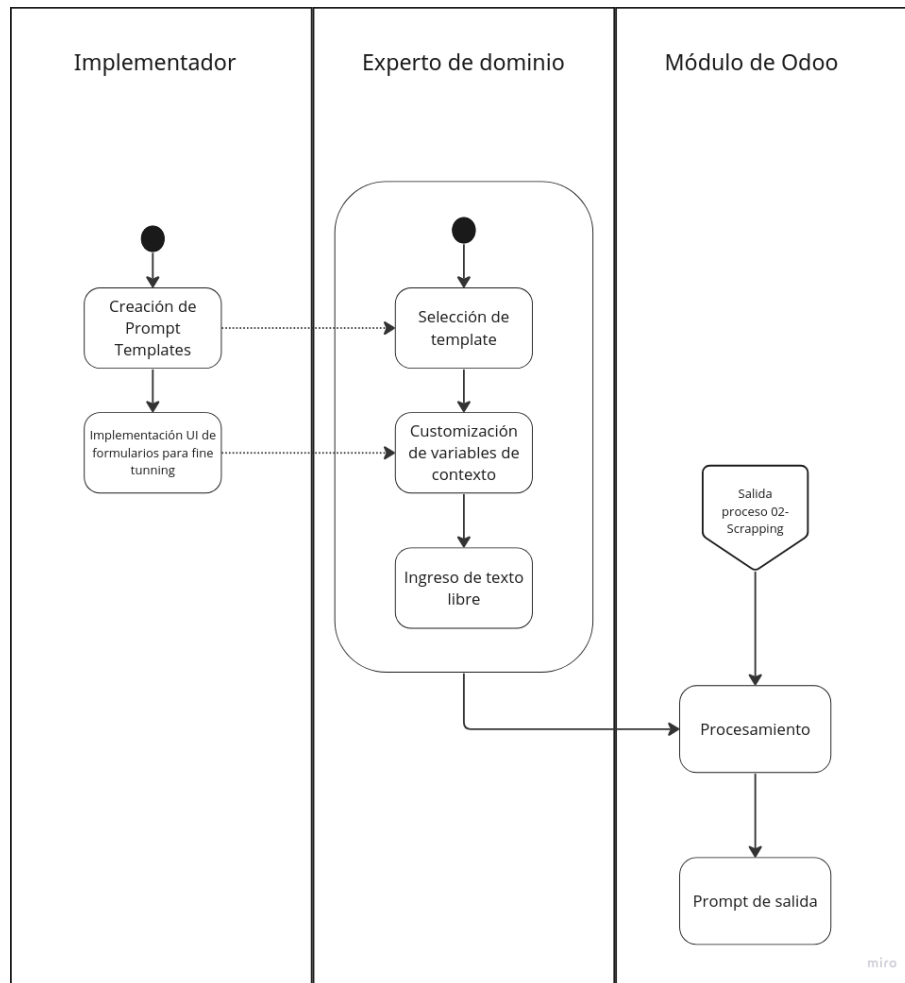
El objetivo de este subproceso es optimizar el embedding de manera personalizada y adaptada al dominio de estudio, lo que puede conducir a una mejora significativa en la calidad y precisión de los resultados generados por la inteligencia artificial generativa.

06. Encoding

Actor: Aplicación de Clustering.

Actividad: Convertir la información recopilada en un formato adecuado para la clusterización. El encoding consiste en representar los datos de manera numérica para poder aplicar algoritmos de clusterización y agruparlos en categorías o conjuntos similares. Esto implica asignar un valor numérico a cada dato o característica relevante que se utilizará en el proceso de agrupación. El proceso de encoding puede involucrar técnicas como la codificación one-hot, donde cada valor posible se convierte en una columna binaria independiente, o la codificación ordinal, donde se asignan valores numéricos a las categorías en función de un orden específico. El objetivo es proporcionar una representación numérica que capture las características y relaciones relevantes de los datos para facilitar su agrupación y análisis.

4.4. Definición de Contexto



Objetivo: Generar el prompt completo que se le va a dar como entrada a la IAG a partir de la integración entre los templates creados de antemano y la intervención del experto de dominio. Este último podrá ajustar variables de entrada y definir el formato de salida esperado para el análisis.

Alcance: Todas las interfaces gráficas necesarias para la toma de datos de los expertos

Entrada: Requerimientos blandos de expertos de dominio y datos empíricos de las noticias

Salida: Prompt completo para la IAG

Inicio

01. Creación de Prompt Templates

Actor: Implementador de la aplicación.

Actividad: Proporcionar una interfaz gráfica que habilite la selección de prompt templates para el experto de dominio. La elección de seleccionar un template en lugar de crear un prompt desde cero se realizó con el objetivo de evitar el “Síndrome de la Página en Blanco” (*Blank Page Syndrome*), el efecto en el que los usuarios, al encontrarse frente a una página en blanco, no saben por dónde comenzar (Zamfirescu-Pereira et al., 2023).

Los templates van a utilizar una lista de instrucciones básicas con características que queremos que todos cumplan basándonos en investigaciones previas sobre cómo crear un prompt adecuado (Zamfirescu-Pereira et al., 2023; Boonstra, 2024). Ej:

- Los prompts deben ser concisos, claros y fáciles de entender. Se debe evitar el lenguaje complejo y la información innecesaria.
- Usar verbos de acción como Actuar, Analizar, Clasificar, Describir, Extraer, Generar, Parsear, Resumir, etc.
- Uso de Instrucciones (qué hacer) en lugar de Restricciones (qué evitar).
- Proporcionar ejemplos de interacciones deseadas en los prompts (One-shot y Few-shot): El one-shot prompting proporciona un solo ejemplo, mientras que el few-shot prompting proporciona múltiples ejemplos. Múltiples ejemplos en few-shot prompting aumentan la probabilidad de que el modelo siga el patrón. Los ejemplos incluidos deben ser relevantes para la tarea, diversos, de alta calidad y bien escritos. Incluir casos extremos (edge cases).
- Escribir prompts que se asemejen (en cierta medida) a código fuente.

La personalización de *prompts* dentro del sistema propuesto no solo se basa en las combinaciones de ajustes que el experto pueda introducir, sino también en la comprensión de las distintas técnicas de *prompting* que permiten optimizar el comportamiento del modelo. En este sentido, los *templates* pueden variar tanto en los resultados esperados como en su estructura, dependiendo del tipo de información que procesen —por ejemplo, texto, imágenes, o una combinación de ambos— y del modelo utilizado (como ChatGPT para texto o DALL · E 2 para imágenes).

Una forma efectiva de categorizar y configurar estos *templates* es mediante la aplicación de estrategias de *prompting* diferenciadas. Distinguir entre estos tipos proporciona un marco conceptual claro para diseñar *prompts* con intención y eficacia (Boonstra, 2024):

- **System Prompting:** Establece el contexto general y el propósito de la tarea. Sirve para definir las capacidades fundamentales del modelo y cómo debe comportarse en términos estructurales. Por ejemplo, se puede usar para solicitar que las respuestas sigan un formato específico, como una estructura en JSON o una tabla.
- **Contextual Prompting:** Proporciona información específica o de fondo que es relevante para la tarea. Este tipo de *prompting* ayuda al modelo a interpretar correctamente la situación, como cuando se incluyen directamente en el *prompt* datos empíricos provenientes de noticias, investigaciones u otras fuentes confiables.
- **Role Prompting:** Asigna una identidad o rol al modelo, moldeando el estilo y tono de la salida. Aunque es menos central para tareas de análisis objetivo, puede ser útil cuando se requiere una narrativa con un enfoque determinado, como la voz de un periodista, un analista político o un divulgador científico.

Así, la construcción de *prompts* eficaces no solo depende del contenido a procesar o del modelo empleado, sino también del diseño intencional del contexto comunicativo. Integrar estos enfoques permite al experto desarrollar *templates* más robustos, flexibles y alineados con los objetivos analíticos del sistema, favoreciendo un mayor control sobre la salida generada por los modelos de lenguaje.

Agregaremos a la interfaz gráfica una forma intuitiva de seleccionar las distintas configuraciones mencionadas.

02. Implementación UI de formularios para fine tuning

Actor: Implementador de la aplicación.

Actividad: Proporcionar una interfaz gráfica que habilite el ajuste de las variables libres que no quedaron definidas en los templates anteriores.

03. Selección de template

Actor: Experto de dominio.

Actividad: Seleccionar un template inicial para el análisis.

04. Customización de variables de contexto

Actor: Experto de dominio.

Actividad: Configuración de las variables necesarias para el *template* seleccionado. En este paso, el experto podrá usar una batería de herramientas de interfaz gráfica para personalizar su contexto, por ejemplo: fechas desde-hasta de publicaciones, palabras clave, combinación de conceptos, etc.

Además de estos filtros contextuales iniciales, esta etapa también incluye la configuración de parámetros clave que afectan el comportamiento del modelo de lenguaje (Boonstra, 2024).

- **Temperature:** Controla el grado de aleatoriedad en la generación de texto. Un valor más bajo (cercano a 0) hace que el modelo sea más determinístico y predecible, lo que resulta útil para tareas fácticas o consistentes, como el análisis de noticias. Valores más altos incrementan la diversidad de la salida, lo cual puede ser útil en tareas creativas. Esta variable ya ha sido contemplada en la tabla del proceso de *scraping*, y forma parte de la definición operativa del contexto para la Inteligencia Artificial Generativa (IAG).
- **Top-K y Top-P:** Son técnicas que restringen la predicción del siguiente token a un subconjunto de tokens con mayor probabilidad. *Top-K* limita la elección a los K tokens más probables, mientras que *Top-P* (también conocido como *nucleus sampling*) considera los tokens cuya probabilidad acumulada alcanza un umbral P. Ambos parámetros permiten modular la aleatoriedad y control de la generación.
- **Output Length:** Define la cantidad máxima de tokens a generar por parte del modelo. Cuantos más tokens se generen, mayor será el cómputo requerido, lo que impacta en el tiempo de respuesta y los recursos utilizados. Esta longitud puede controlarse directamente en el sistema o bien incluirse como una instrucción dentro del *prompt*. La tesis ya hace referencia al “Token Limit” dentro de las tablas de ejemplo de *prompts*.
- **Uso de variables:** Para aumentar la reutilización y adaptabilidad de los *templates*, es recomendable incorporar variables dinámicas en los *prompts*. Estas permiten insertar datos contextuales (como fechas,

actores, ubicaciones o temas) sin necesidad de reescribir el *prompt* completo cada vez. Esta estrategia, también mencionada previamente en la tesina, optimiza el proceso de personalización y facilita la integración del sistema en flujos de trabajo automatizados.

05. Ingreso de texto libre

Actor: Experto de dominio.

Actividad: El experto podrá ingresar libremente aclaraciones o ejemplos que querrá incluir en el prompt para dejar en claro cual es su idea de análisis. Con el fin de guiar de manera efectiva el proceso de generación, resulta útil incluir ejemplos o escenarios específicos en los prompts. Estos ejemplos pueden ilustrar los tipos de interacciones o respuestas deseadas que la IAG debería generar. Al incorporarlos la IAG puede aprender y emular el comportamiento deseado.

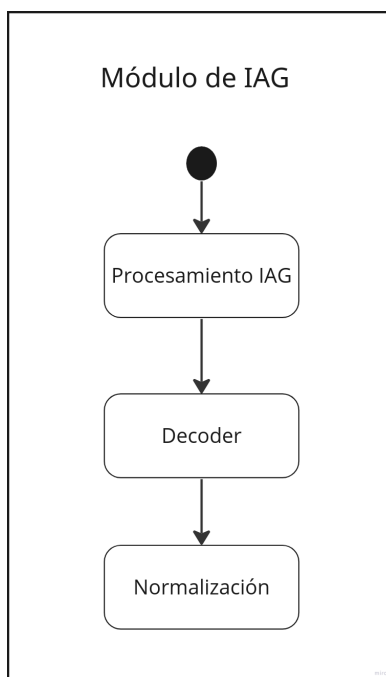
En un estudio previo todos los participantes mostraron una fuerte preferencia por la instrucción directa en lugar de proporcionar ejemplos en línea, y cuando se les animó a dar ejemplos, la mayoría de los participantes lo hicieron y señalaron su efectividad en la mejora de resultados (Zamfirescu-Pereira et al., 2023).

06. Procesamiento

Actor: Módulo de Odoo.

Actividad: Estructurizar el prompt que sera entregado a la IAG utilizando las entradas del experto y los datos empíricos de las noticias. Se espera la salida de una string formateada en algún lenguaje de marcado que sea interpretado por la IAG.

4.5. Procesamiento IAG



01. Procesamiento IAG

Actor: Aplicación de Inteligencia Artificial Generativa.

Actividad: Utilizando los datos previamente descriptos, el prompt adecuadamente construido y una conexión estable con la API del modelo generativo seleccionado, se ejecuta la petición correspondiente. El objetivo principal de esta actividad es generar resultados creativos y originales a partir de la inteligencia artificial generativa.

02. Decoder

Actor: Aplicación de Decodificación.

Actividad: Preparar la información generada por la IAG para su posterior reutilización en otros contextos o plantillas.

03. Normalización

Actor: Investigador o analista de datos.

Actividad: Realizar un racconto final de todo el camino recorrido durante el proceso, desde los datos iniciales hasta la generación de la salida del modelo

generativo. El objetivo de esta actividad es realizar una normalización de los datos y resultados obtenidos para poder realizar comparaciones, evaluaciones y análisis posteriores de manera coherente y sistemática.

La normalización implica varias tareas, como:

- Documentación detallada: documentar todo el proceso en un informe o documento, donde se especifican todas las etapas, herramientas utilizadas, parámetros, ajustes y cualquier otro detalle relevante. Esta documentación sirve como referencia para futuros análisis y también permite una mayor transparencia y reproducibilidad del proceso.
- Organización de los datos: organizar datos recopilados, los parámetros utilizados, el prompt generado y cualquier otra información relevante en una estructura coherente y accesible. Esto facilita la revisión y comparación de los datos en etapas posteriores.
- Comparación con variables: al definirse variables específicas, el investigador realiza una comparación de la salida generada por el modelo con estas variables. Esto permite evaluar qué tan bien se ajusta la salida a las expectativas y criterios establecidos, y también identificar posibles desviaciones o inconsistencias.
- Análisis de resultados: analizar en función de los criterios y objetivos deseados. Esto puede incluir la evaluación de la coherencia del texto generado, la calidad de las respuestas, la relevancia semántica o cualquier otro aspecto específico relacionado con el proyecto de investigación.

4.6. Visualización previa

4.7. Refinamiento iterativo

4.8. Presentación de resultados

4.9. Síntesis del capítulo

Capítulo 5

Ejemplo de aplicación

- 5.1. Selección del caso
- 5.2. Configuración inicial
- 5.3. Ejecución del scraping
- 5.4. Definición de contexto y template utilizado
- 5.5. Resultados del análisis generativo
- 5.6. Discusión

Capítulo 6

Conclusiones y Trabajo a Futuro

Apéndice A

Tipos de datos del pipeline

La especialización de tubos tipados adoptada en el Capítulo 3 exige que cada tubo declare el tipo del *recordset* que transporta (Subsección 3.2.4). El capítulo nombra esos tipos —**Noticia**, **Embedding**, **Prompt**, **RespuestaIAG**, **Análisis**, **Veredicto** y **Resultado**— y describe las transformaciones que los producen; este apéndice completa la caracterización especificando la estructura interna de cada uno. Las definiciones constituyen los contratos de datos del framework: fijan qué produce cada filtro y qué puede esperar el siguiente, con independencia de la plataforma sobre la que el framework se materialice. Se especifica además la estructura del *template* —la entidad del repositorio que ancla la definición del estudio—, que no circula por los tubos pero es referida por los tipos del flujo (Sección A.9).

A.1. Notación y criterios de diseño

Cada tipo se especifica como un registro de pares **clave: tipo**, en una notación de estilo JSON. Los tipos primitivos son **string**, **int**, **float** y **datetime**; **float[]** denota una lista de valores y **objeto** una estructura anidada cuyo esquema no fija el framework. El marcador **ref** denota una referencia a un registro del repositorio común —el comentario de cada clave indica el registro referenciado— y **ref []**, una colección ordenada de referencias; esta representación es la contraparte, a nivel del dato, del transporte por referencia que caracteriza a los conectores del sistema (Subsección 3.2.5). El sufijo **?** marca una clave opcional. La cardinalidad con que cada tipo circula por los tubos —**Tipo[c]**— se detalló en la Subsección 3.2.3 y se recuerda al inicio de cada

sección.

Tres criterios, aplicados de manera uniforme, gobiernan qué claves integran cada contrato y cuáles se excluyen deliberadamente.

1. **Contrato mínimo.** Cada tipo contiene únicamente la carga útil que algún filtro posterior consume y la procedencia del dato —cuándo se produjo y a partir de qué registros—. Toda clave adicional volvería al contrato más exigente de lo que el pipeline requiere, dificultando la sustitución de los filtros que lo producen.
2. **La procedencia viaja; la identidad del productor, no.** Registrar a partir de qué dato se produjo un registro es procedencia, y sostiene la auditabilidad; registrar con qué recurso técnico se lo produjo —el modelo de *embedding*, el proveedor generativo— describiría la identidad del filtro productor, que el estilo veda conocer a los restantes filtros (Subsección 3.1.2). Cambiar ese recurso equivale a sustituir el filtro, no a alterar los datos que emite.
3. **Los esquemas de dominio se delegan al estudio.** Donde el contenido depende del caso analítico —**Análisis y Resultado**—, el framework fija el sobre (procedencia y carga estructurada) y delega el esquema interno a la definición del estudio, declarada en el *template* (Sección A.9). Fijar claves de dominio en el contrato casaría al framework con un caso de estudio particular, contra el requisito de diversidad analítica (Subsección 3.1.3).

A.2. Noticia

Producido por Extracción; circula como `Noticia[N]` por el tubo $2 \rightarrow 3$, donde N es el tamaño del corpus.

```

Noticia {
  titulo:          string    // título de la publicación
  copete?:         string    // resumen editorial del portal
  texto:           string    // cuerpo completo, normalizado
  link:            string    // URL canónica; clave única
  medio:           ref       // portal de origen
  departamento:    string    // zona geográfica del portal
  fecha_hora?:     datetime  // fecha declarada de publicación
  fecha_registro:  datetime  // fecha de ingreso al corpus
  reglas_aplicadas: string    // reglas que la seleccionaron
}

```

Las cuatro primeras claves constituyen el contenido extraído: el título, el copete —el resumen editorial que acompaña al título—, el cuerpo textual completo y el enlace canónico de la publicación, que oficia además de identificador único del registro en el repositorio. Las restantes registran la procedencia: el portal del que la noticia fue extraída, la adscripción geográfica de ese portal, la fecha de publicación que el portal declara —opcional, pues no todo portal la expone en forma interpretable—, la fecha cierta de incorporación al corpus y las reglas de extracción que motivaron su selección. Esta última clave completa la trazabilidad de la etapa: documenta por qué cada noticia integra el corpus, en términos de las reglas configuradas en el sintonizador de Extracción. El copete comparte la opcionalidad de la fecha de publicación: su disponibilidad depende de lo que el portal publique y de la vía de extracción empleada.

A.3. Embedding

Producido por Enriquecimiento semántico; circula como `Embedding[N]` por el tubo interno 4→5 del filtro compuesto Recuperación aumentada.

```

Embedding {
  noticia:         ref        // noticia que codifica
  vector:          float[]    // representación vectorial
  fecha_registro:  datetime   // fecha de cómputo del vector
}

```

El tipo reifica la transformación de Enriquecimiento semántico: cada registro asocia una noticia —por referencia— con su representación vectorial, de modo que el filtro emite las representaciones y no las noticias que las

originan (Subsección 3.2.3). El contrato ilustra el segundo criterio de la Sección A.1: no incluye un identificador del modelo que computó el vector, porque ese dato describiría la identidad del filtro productor y no la procedencia del dato. Dos vectores sólo son comparables dentro de un mismo espacio semántico; reemplazar el modelo de *embedding* equivale, por lo tanto, a sustituir el filtro completo y recomputar las representaciones del corpus, no a mezclar en el repositorio vectores de espacios heterogéneos.

A.4. Prompt

Producido por Composición de contexto; circula como **Prompt** [M] por el tubo 6→7, donde M es la cantidad de unidades de análisis del estudio.

```
Prompt {
  texto:          string      // prompt completo para el modelo
  parametros: {          // parámetros de generación
    temperatura:    float
    longitud_salida: int
  }
  noticias:       ref[]      // noticias del contexto (M:N)
  template:       ref        // template de origen
  fecha_registro: datetime  // fecha de composición
}
```

La clave **texto** materializa la composición completa —*template*, variables de contexto, aclaraciones de texto libre y material empírico recuperado—, por lo que el registro documenta en forma íntegra lo que se envía al modelo generativo; los ingredientes de la composición no se registran por separado, pues su combinación está ya materializada en el texto y el *template* de origen —con el objetivo del análisis y el esquema de respuesta que éste declara (Sección A.9)— queda anclado por referencia. La clave **parametros** merece una precisión: la temperatura y la longitud de salida las fija el experto en el sintonizador de Composición, pero su consumidor es Invocación generativa; dado que los sintonizadores no se invocan entre filtros (Subsección 3.2.6), la única vía compatible con el estilo para hacérselos llegar es el propio dato. No se trata de configuración del filtro filtrándose en el contrato, sino de información que un consumidor aguas abajo espera. La clave **noticias**, por último, registra la relación de muchos a muchos entre noticias y *prompts* que la recuperación semántica establece (Subsección 3.2.3).

A.5. RespuestaIAG

Producido por Invocación generativa; circula como **RespuestaIAG**[M] por el tubo 8→9.

```
RespuestaIAG {  
  prompt:      ref      // prompt que la originó  
  texto:       string   // respuesta cruda, sin decodificar  
  fecha_registro: datetime // fecha de recepción  
}
```

El registro conserva la respuesta cruda tal como la produjo el servicio generativo, sin interpretación alguna: su decodificación corresponde al filtro siguiente. Por el segundo criterio de diseño quedan fuera el proveedor, el modelo y sus credenciales —identidad del filtro de Invocación, cuya sustitución completa es precisamente lo que el aislamiento de la etapa habilita (Subsección 3.2.2)—, así como la telemetría operacional de la petición, que ningún filtro consume.

A.6. Análisis

Producido por Decodificación; circula como **Análisis**[M] por el tubo 10→11.

```
Análisis {  
  respuesta:      ref      // respuesta cruda de origen  
  contenido:      objeto   // estructura analítica; esquema  
                      // declarado por el template  
  fecha_registro: datetime // fecha de decodificación  
}
```

El tipo aplica el tercer criterio de diseño: el framework fija el sobre y delega el esquema. La clave **contenido** porta la estructura analítica ya validada y canonicalizada, cuyo esquema interno —qué claves, con qué tipos y vocabularios— lo declara el *template* del estudio (Sección A.9): la definición analítica que declara el objetivo del análisis y especifica qué se le pide al modelo especifica también, como su contraparte, qué estructura se espera de la respuesta. El **contenido** queda así plenamente caracterizado: es una instancia del **esquema_respuesta** del *template*, producida bajo el **objetivo** que éste declara. Esta delegación explica, además, cómo Decodificación valida sin exponer sintonizador: alcanza el esquema esperado siguiendo la cadena de referencias que va de la respuesta a su *prompt* y de éste a su *template*

(Subsección 3.2.2). El tipo no incluye un estado de validación: la validación estructural es una compuerta y no un atributo, de modo que toda instancia de **Análisis** que circula por el pipeline es, por construcción, una estructura conforme. Tampoco incluye un juicio sobre la fidelidad de su contenido a la evidencia: esa valoración —de otra naturaleza que la conformidad estructural— la produce el filtro de Verificación aguas abajo y se reifica en un registro **Veredicto** separado (Sección A.7), no como una clave de este contrato.

A.7. Veredicto

Producido por Verificación; circula como **Veredicto**[M] por el tubo 12→13.

```
Veredicto {
  analisis:      ref      // análisis juzgado
  estado:        string   // válido | observado | no sustentado
  confianza:     float    // fidelidad del análisis a la evidencia
  sustento:      objeto   // evidencia que respalda o refuta cada
                          // afirmación; esquema según el
  criterio
                          // de validez del estudio
  fecha_registro: datetime // fecha de verificación
}
```

El tipo reifica el juicio de fidelidad que produce Verificación: cada registro asocia un análisis —por referencia— con la valoración de en qué medida sus afirmaciones están sostenidas por la evidencia empírica de la que dicen derivar (Subsección 3.2.2). La clave **estado** porta el dictamen canónico —válido, observado o no sustentado—; **confianza** lo cuantifica, reificando a nivel del dato la calibración de incertidumbre que la revisión sistemática identificó como brecha (Sección 2.4). Que ambas claves acompañen al veredicto, y que ninguno de los análisis se descarte, es lo que materializa la *compuerta blanda*: el filtro califica y señala, pero no suprime, y deja la decisión editorial última al analista. La clave **sustento** aplica el tercer criterio de diseño —delega al estudio el esquema de dominio—: su estructura interna —qué fragmentos se citan y cómo se los pondera— depende del criterio de validez configurado en el sintonizador de Verificación, no del framework. Por el segundo criterio, en cambio, el contrato omite deliberadamente la identidad del ejecutor de la verificación —el procedimiento automático, el revisor humano asistido o

el conjunto de agentes que confrontan sus juicios—: registrar con qué recurso se produjo el veredicto describiría la identidad del filtro productor, cuya sustitución es precisamente lo que la agnosticidad del ejecutor habilita (Subsección 3.1.3), del mismo modo que **Embedding** omite el modelo que computó el vector y **RespuestaIAG**, el proveedor generativo.

A.8. Resultado

Producido por Normalización; se entrega por el puerto 14 al sumidero como registro único, tipado sin cardinalidad.

```
Resultado {
  veredictos:      ref[]      // veredictos consolidados
  contenido:       objeto     // producto consolidado; esquema
                        // según criterios del estudio
  fecha_registro: datetime    // fecha de consolidación
}
```

Como **Análisis**, el tipo fija el sobre y delega el esquema: la forma del producto consolidado —distribuciones temáticas, comparaciones entre portales, series temporales— depende del caso analítico y la definen los criterios de consolidación del estudio. La clave de procedencia **veredictos** referencia los veredictos que Normalización consolidó —y, a través de ellos, los análisis verificados que estos juzgan—, de modo que el juicio de fidelidad queda incorporado a la cadena de procedencia del producto final (Sección A.11). Esos criterios, así como las variables de comparación del analista, no integran el contrato: son configuración del sintonizador de Normalización consumida por el propio filtro, y su efecto queda materializado en el contenido consolidado.

A.9. Template

A diferencia de los siete tipos anteriores, el *template* no circula por los tubos: es la entidad del repositorio común que ancla la definición del estudio, creada y editada por el experto del dominio a través del panel de *templates* (Subsección 3.2.6) e incorporada al flujo por referencia —cada **Prompt** la ancla directamente y cada **Análisis** la alcanza por la cadena de referencias—.

```

Template {
  nombre:          string    // identificación de la plantilla
  objetivo:        string    // objetivo del análisis: alcance,
                           // paradigma, referencias
  instrucciones:   string    // cuerpo del prompt, con variables
  variables:       objeto    // variables de contexto declaradas
  esquema_respuesta: objeto  // esquema esperado del contenido
}

```

El tipo reifica los tres datos de entrada para la construcción de *prompts* anticipados en el Capítulo 1: los datos empíricos ingresan por la recuperación semántica en Composición de contexto; los objetivos requeridos para el análisis —alcance, paradigma de análisis, referencias a tener en cuenta— se declaran en la clave **objetivo**; y el tipo de salida o representación se fija en **esquema_respuesta**, el esquema contra el cual valida Decodificación. La clave **objetivo** merece énfasis: el pipeline no transporta únicamente datos empíricos transformados, sino también la intención analítica que les da sentido —bajo qué paradigma, con qué alcance y contra qué referencias se analiza—, que viaja por referencia con cada **Prompt** y cada **Análisis**, del mismo modo que todo contenido del sistema. Por tratarse de configuración editada por sintonizador, los registros **Template** quedan fuera del alcance de la monotonía del flujo (Sección A.10); la referencia ancla la identidad de la plantilla, y la composición íntegra efectivamente enviada al modelo queda materializada en el texto de cada **Prompt** (Sección A.4), lo que preserva la auditabilidad de lo ejecutado aun si la plantilla es editada con posterioridad.

A.10. Los filtros como transformaciones de estado

Definidos los tipos, las especificaciones funcionales del Capítulo 3 admiten una síntesis formal. Según la formulación introducida en la Subsección 3.2.2 —que extiende la notación funcional del catálogo de Cristiá and Pomponio (2025, p. 24) a la variante heterogénea—, cada filtro realiza una transformación $f : (T[c], Rep) \rightarrow (T'[c'], Rep')$, donde $Rep' = Rep \cup \Delta_f$, Δ_f es el conjunto de registros que el filtro persiste y el *recordset* emitido es $refs(\Delta_f)$. Las siete firmas del pipeline se encadenan:

Extracción:	$(\text{flujo del portal}, Rep_0) \rightarrow (\text{Noticia}[N], Rep_1)$
Enriquecimiento:	$(\text{Noticia}[N], Rep_1) \rightarrow (\text{Embedding}[N], Rep_2)$
Composición:	$(\text{Embedding}[N], Rep_2) \rightarrow (\text{Prompt}[M], Rep_3)$
Invocación:	$(\text{Prompt}[M], Rep_3) \rightarrow (\text{RespuestaIAG}[M], Rep_4)$
Decodificación:	$(\text{RespuestaIAG}[M], Rep_4) \rightarrow (\text{Análisis}[M], Rep_5)$
Verificación:	$(\text{Análisis}[M], Rep_5) \rightarrow (\text{Veredicto}[M], Rep_6)$
Normalización:	$(\text{Veredicto}[M], Rep_6) \rightarrow (\text{Resultado}, Rep_7)$

con $Rep_i = Rep_{i-1} \cup \Delta_i$, donde cada Δ_i es el conjunto de registros del tipo correspondiente que la etapa persiste: los N registros **Noticia**, los N **Embedding**, los M **Prompt**, y así sucesivamente. El encadenamiento de los subíndices exhibe el enhebrado del repositorio a través del pipeline: cada filtro recibe el estado que dejó el anterior. La monotonía $Rep_0 \subseteq Rep_1 \subseteq \dots \subseteq Rep_7$ se afirma sobre los registros del flujo de datos: los sintonizadores editan configuración (*templates*, reglas de extracción, criterios de validez y de consolidación), pero esa configuración no pertenece al flujo ni es modificada por ningún filtro. El estado final Rep_7 contiene, por lo tanto, la historia completa del estudio: el corpus, sus representaciones semánticas, los *prompts* compuestos, las respuestas crudas, los análisis decodificados, los veredictos de verificación y el resultado consolidado.

Por el cierre composicional del estilo (Subsección 3.1.1), la composición de las siete firmas es a su vez un filtro en el sentido extendido: el sistema completo realiza la transformación $(\text{flujo del portal}, Rep_0) \rightarrow (\text{Resultado}, Rep_7)$, que convierte el flujo de los portales digitales en un producto analítico consolidado y deja persistida en el repositorio la historia íntegra de esa conversión. Sobre esa historia se construye la cadena de procedencia que se describe a continuación.

A.11. La cadena de procedencia

Leídas en conjunto, las claves de procedencia de los siete tipos forman una cadena continua de referencias que recorre el pipeline en sentido inverso:

Resultado $\rightarrow$ **Veredicto** $\rightarrow$ **Análisis** $\rightarrow$ **RespuestaIAG** $\rightarrow$ **Prompt** $\rightarrow$
Noticia (y *template*)

Desde el producto final del estudio puede así reconstruirse qué veredictos de verificación lo componen, sobre qué análisis se pronuncia cada uno, de

qué respuestas crudas provienen esos análisis, con qué *prompts* —y bajo qué *template* y parámetros— se generaron, sobre qué noticias se apoyó cada composición y, para cada noticia, de qué portal fue extraída, cuándo, y qué reglas motivaron su ingreso al corpus; la rama **Embedding** → **Noticia** completa el cuadro para la capa de recuperación. La documentación del recorrido del proceso que la especificación de Normalización promete (Subsección 3.2.2) no requiere, por lo tanto, un registro adicional: es una propiedad estructural del sistema de tipos, y constituye la materialización, a nivel del dato, de la auditabilidad lineal que motivó la elección del estilo (Subsección 3.1.3).

Bibliografía

- Hamza Ameer. NexGen MediaCraft: Real-time AI news anchoring with LLM-based generation and web scraping. Zenodo, 2025. URL <https://doi.org/10.5281/zenodo.15856484>.
- Apache Software Foundation. XComs — apache airflow documentation, 2026. URL <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/xcoms.html>. Consultado en junio de 2026.
- Nafeesa Begum J and Chandru D. An automated daily news reports generating application involving keyword-based news scraping, summarization, and sentimental analysis leveraging NLP models. *International Journal for Research in Applied Science and Engineering Technology*, 12(11): 1556–1565, 2024. doi: 10.22214/ijraset.2024.65443.
- Amir Behbahanian. GlimzAI: A fully automated AI-powered news aggregation, predictive polling, and content distribution platform using LLM-based agentic workflows. ResearchGate publication 385661008, s.f. URL <https://www.researchgate.net/publication/385661008>. Fecha de publicación no verificable; ResearchGate restringe el acceso. La cita usa “s.f.” por convención conservadora.
- D. Berry. The computational turn: thinking about the digital humanities. *Culture Machine*, Vol. 12, 2011.
- Franck Bertagnolio, Michaela Herr, and Kaj Dam Madsen. A roadmap for required technological advancements to further reduce onshore wind turbine noise impact on the environment. 2023.
- Lee Boonstra. Prompt engineering. *White paper by Google*, 2024.

- Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas C. Müller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake Vanderplas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. API design for machine learning software: Experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013. URL <https://arxiv.org/abs/1309.0238>.
- Carlón. Circulación del sentido y construcción de colectivos en una sociedad hipermediatizada. *Nueva Editorial Universitaria, San Luis*, 2020.
- N. Couldry and A. Hepp. The continuing lure of the mediated centre in times of deep mediatization: Media events and its enduring legacy. *Media, Culture & Society*, 40(1), 2017.
- Maximiliano Cristiá and Laura Pomponio. Catálogo incompleto de estilos arquitectónicos. Apunte de cátedra, Ingeniería de Software 2, Facultad de Ciencias Exactas, Ingeniería y Agrimensura, Universidad Nacional de Rosario, 2025.
- David Garlan and Mary Shaw. An introduction to software architecture. Technical Report CMU-CS-94-166, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, January 1994. URL https://www.cs.cmu.edu/afs/cs/project/able/ftp/intro_softarch/intro_softarch.pdf. También publicado como CMU Software Engineering Institute Technical Report CMU/SEI-94-TR-21, ESC-TR-94-21.
- Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. arXiv:2501.10868 [cs.CL], 2025. URL <https://arxiv.org/abs/2501.10868>.
- I. Gindin and M. Busso. Investigaciones en comunicación en tiempos de big data: sobre metodologías y temporalidades en el abordaje de redes sociales. *Revista Científica de Estrategias, Tendencias e Innovación en Comunicación*, n^o15. 25-43., 2018.

- Samar Haider, Amir Tohidi, Jenny S. Wang, Timothy Dörr, David M. Rothschild, Chris Callison-Burch, and Duncan J. Watts. The media bias detector: A framework for annotating and analyzing the news at scale. *arXiv preprint arXiv:2509.25649*, 2025. doi: 10.48550/arXiv.2509.25649. URL <https://arxiv.org/abs/2509.25649>.
- Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, Boston, MA, 2003. ISBN 978-0321200686.
- A. Hotho, A. Nürnberger, and G. Paaß. A brief survey of text mining. *Ldv Forum*, Vol. 20. 19–62, 2005.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Niklas Luhmann. The reality of the mass media. *Stanford University Press*, 2000.
- Maxwell McCombs, Juan Pablo Llamas, Esteban Lopez-Escobar, and Federico Rey. Candidate images in Spanish elections: Second-level agenda-setting effects. *Journalism & Mass Communication Quarterly*, 74(4):703–717, 1997. doi: 10.1177/107769909707400404.
- Maxwell E. McCombs and Donald L. Shaw. The agenda-setting function of mass media. *The Public Opinion Quarterly*, 36(2):176–187, 1972.
- J-G Meunier. Computational semiotics. *Londres: Bloomsbury*, 2022.
- Microsoft. Pipes and filters pattern — azure architecture center, 2025. URL <https://learn.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters>. Consultado en junio de 2026.
- Jeremiah Milbauer, Ziqi Ding, Zhijin Wu, and Tongshuang Wu. NewsSense: Reference-free verification via cross-document comparison. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 422–430, Singapore, 2023. Association

- for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-demo.39. URL <https://aclanthology.org/2023.emnlp-demo.39/>.
- Amul Neupane, Kapalik Khanal, Nitesh Nepal, and Naseeb Dangi. Advanced news aggregation and content generation using LLMs and NLP algorithms. *European Journal of Applied Science, Engineering and Technology*, 3(2): 295–303, March 2025. ISSN 2786-9342. doi: 10.59324/ejaset.2025.3(2).24. URL <https://ieeexplore.ieee.org/abstract/document/11079577>.
- Odoo S.A. ORM API. Odoo 17.0 Developer Documentation, 2024. URL <https://www.odoo.com/documentation/17.0/developer/reference/backend/orm.html>. Consultado en junio de 2026.
- Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, Roger Chou, Julie Glanville, Jeremy M. Grimshaw, Asbjørn Hróbjartsson, Manoj M. Lalu, Tianjing Li, Elizabeth W. Loder, Evan Mayo-Wilson, Steve McDonald, Luke A. McGuinness, Lesley A. Stewart, James Thomas, Andrea C. Tricco, Vivian A. Welch, Penny Whiting, and David Moher. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, 372:n71, 2021. doi: 10.1136/bmj.n71. URL <https://www.bmj.com/content/372/bmj.n71>.
- Bohdan M. Pavlyshenko. Using GPT models for qualitative and quantitative news analytics in the 2024 US presidential election process. *arXiv preprint arXiv:2410.15884*, 2024. URL <https://arxiv.org/abs/2410.15884>.
- L. Prades, F. Romero, A. Estruch, A. García-Domínguez, and J. Serrano. Defining a methodology to design and implement business process models in bpmn according to the standard ansi/isa-95 in a manufacturing enterprise. *Procedia Engineering*, 63:115–122, 2013. ISSN 1877-7058. doi: <https://doi.org/10.1016/j.proeng.2013.08.283>. URL <https://www.sciencedirect.com/science/article/pii/S1877705813014963>. The Manufacturing Engineering Society International Conference, MESIC 2013.
- Pablo Rey-Mazón. *COLOR OF CORRUPTION. Visual evidence of agenda-setting in a complex mass media ecosystem*. PhD thesis, Universitat Oberta de Catalunya, 2023.

- Daniel Ritter. Experiences with business process model and notation for modeling integration patterns. In Jordi Cabot and Julia Rubin, editors, *Modelling Foundations and Applications (ECMFA 2014)*, volume 8569 of *Lecture Notes in Computer Science*, pages 254–266, Cham, 2014. Springer. doi: 10.1007/978-3-319-09195-2_17. Versión técnica extendida disponible como arXiv:1403.4053.
- Jim Samuel, Tanya Khanna, and Srinivasaraghavan Sundar. The rise of artificial intelligence phobia! unveiling news-driven spread of AI fear sentiment using ML, NLP, and LLMs. *IEEE Access*, 13:125944–125969, 2025. doi: 10.1109/ACCESS.2025.3588179.
- Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, Upper Saddle River, NJ, 1996. ISBN 978-0131829572.
- Somasundharam et al. Personalized news aggregation with AI: Leveraging LLMs, user data, and portfolio information for automated content creation. DiVA Portal, 2025. URL <https://www.diva-portal.org/smash/get/diva2:1978319/FULLTEXT01.pdf>. Autores secundarios pendientes de completar al acceder al texto completo.
- A. Tan. Text mining: the state of the art and the challenges. *En Proceedings Pakdd’99 workshop on knowledge discovery from advanced databases.*, 1999.
- The Kubeflow Authors. Pipelines: Artifacts — kubeflow documentation, 2026. URL <https://www.kubeflow.org/docs/components/pipelines/user-guides/data-handling/artifacts/>. Consultado en junio de 2026.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Gian Wiher, Clara Meister, and Ryan Cotterell. On decoding strategies for neural text generators. *Transactions of the Association for Computational Linguistics*, 10:997–1012, 2022. doi: 10.1162/tacl_a_00502.
- Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. arXiv:2307.09702 [cs.CL], 2023. URL <https://arxiv.org/abs/2307.09702>.

J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. Why johnny can't prompt: How non-ai experts try (and fail) to design llm prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394215. doi: 10.1145/3544548.3581388. URL <https://doi.org/10.1145/3544548.3581388>.